

MEMENTO: Memory-Guided Memetic Code-as-Policy Evolution

Alkis Sygkounas

alkis.sygkounas@oru.se

Center for Applied Autonomous Sensor Systems (AASS), Örebro University, Sweden

Victor Aregbede

victor.aregbede@oru.se

Center for Applied Autonomous Sensor Systems (AASS), Örebro University, Sweden

Amy Loutfi

amy.loutfi@oru.se

Örebro University, 70281 Örebro, Sweden and Linköping University, 58183 Linköping, Sweden

Andreas Persson

andreas.persson@oru.se

Center for Applied Autonomous Sensor Systems (AASS), Örebro University, Sweden

Abstract

Long-horizon embodied tasks require policies that execute many dependent actions before task success can be observed. Representing policies as executable control programs (code-as-policy) enables their decision logic to be inspected and revised after rollout evaluation. Revised programs can then be executed and compared by rollout performance, framing policy improvement as execution-guided program search. Evolutionary methods driven by large language models (LLMs) provide a natural mechanism for this search by generating variants and selecting high-performing candidates. However, existing approaches primarily select among independently generated variants and lack a sequential local improvement phase. We introduce MEMENTO, a memory-guided single-elite memetic framework for code-as-policy evolution. MEMENTO first evolves a rollout evaluator that maps policy rollouts to scalar fitness and structured feedback metrics. Fitness selects accepted candidates and the next elite, while feedback metrics condition policy proposals generated by memory-guided hill-climbing, macro-mutation, and crossover. We evaluate MEMENTO on two long-horizon embodied domains: Robosuite Franka Tower-of-Hanoi manipulation and AI2-THOR household interaction. MEMENTO outperforms Eureka and REvolve, adapted as code-as-policy evolutionary baselines, in task success and generalization to held-out Robosuite object configurations and unseen AI2-THOR scenes. Ablations show that zero-shot generation and unevolved evaluators fail to solve either domain, and that removing policy-search branches reduces performance. Finally, we deploy the best-evolved Robosuite policy on a physical Franka robot, demonstrating the feasibility of sim-to-real transfer of the evolved code-as-policy. Code, prompts, and videos are available at: <https://github.com/sygkounas/MEMENTO>.

Keywords

Memetic algorithms, genetic programming, code-as-policy, large language models, sim-to-real transfer.

1 Introduction

Long-horizon embodied control tasks require policies to complete many dependent actions before task success can be evaluated (Mees et al., 2022; Shridhar et al., 2020; Srivastava et al., 2022). Earlier actions determine the task state encountered by later

actions (Shridhar et al., 2020; Srivastava et al., 2022). Correct action order is therefore insufficient; each action must also be physically executed from the observed task state. This creates a long-horizon credit-assignment problem because task success is observed only after many dependent actions (Arjona-Medina et al., 2019). Existing approaches reduce the effect of delayed feedback by adding external structure to policy learning or by changing how experience is generated and credited. Demonstration-based and hierarchical methods can constrain long-horizon behavior through trajectories, subgoal orderings, or reusable skills, while reinforcement-learning methods make delayed task feedback more usable through exploration, relabeling, temporal abstraction, or return redistribution (Zhao et al., 2023; Wang et al., 2023a; Shridhar et al., 2020; Padmakumar et al., 2022; Andreas et al., 2017; Andrychowicz et al., 2017; Pathak et al., 2017; Ecoffet et al., 2021; Sutton et al., 1999; Arjona-Medina et al., 2019). These approaches improve long-horizon learning, but they do not directly expose the decision logic of a failed policy or identify which part of that logic should be revised.

When policies are represented as executable programs, decision logic is written directly as program code and can thereby be inspected and revised. Large language models (LLMs) generate robot policy code that processes perception outputs and parameterizes control-primitive APIs, a formulation termed *code-as-policy* (Liang et al., 2023). This formulation has been extended to contact-rich control and video-conditioned policy generation (Burns et al., 2024; Xie et al., 2025). In long-horizon tasks, however, a generated program can execute without errors while its control logic fails to achieve the task (Liu et al., 2024), and failure feedback from rollouts has, therefore, been used to characterize policy errors and guide refinement (Ying et al., 2025). *Code-as-policy* improvement can then be formulated as execution-guided program search, in which candidate policy programs are executed in the task environment, evaluated based on observed behavior, and revised in response to failure feedback. Binary task success alone, however, is too sparse to guide the revision of a long-horizon policy program, and structured rollout feedback indicating which policy decisions to revise is difficult to specify by hand.

In genetic programming, candidate programs are executed and assigned a fitness score based on performance, which is then used to select programs for variation by micro-mutations that make small local changes, macro-mutations that modify larger program components, and crossover (Koza, 1992; Brameier and Banzhaf, 2007; Angeline, 1997). High-performing programs often require many candidate evaluations (Echevarrieta et al., 2025), making undirected syntactic variation costly in rollout-evaluated embodied domains. Language models have since been used as variation operators over executable programs, replacing syntactic program modifications with prompt-conditioned code proposals (Lehman et al., 2023; Hemberg et al., 2024). LLM-driven code evolution has also been applied to executable policy programs, where candidate controllers are evaluated in task environments and selected for further revision (Hu et al., 2026). In embodied control, LLM-guided evolutionary methods have evolved reward code via LLM-generated variation, with candidate reward functions selected based on policy performance (Ma et al., 2024; Hazra et al., 2025). However, existing *code-as-policy* and reward-code evolution methods primarily select among independently generated variants; they neither refine an accepted candidate through a local-search chain nor retain a memory of revisions that have already failed. Memetic algorithms support this form of refinement by combining evolutionary variation with local search on selected candidates (Moscato, 1989; Neri and Cotta, 2012). The local-search component enables exploitation of an accepted candidate via iterative pro-

gram revision, while mutation and crossover introduce additional variation via larger changes and recombination.

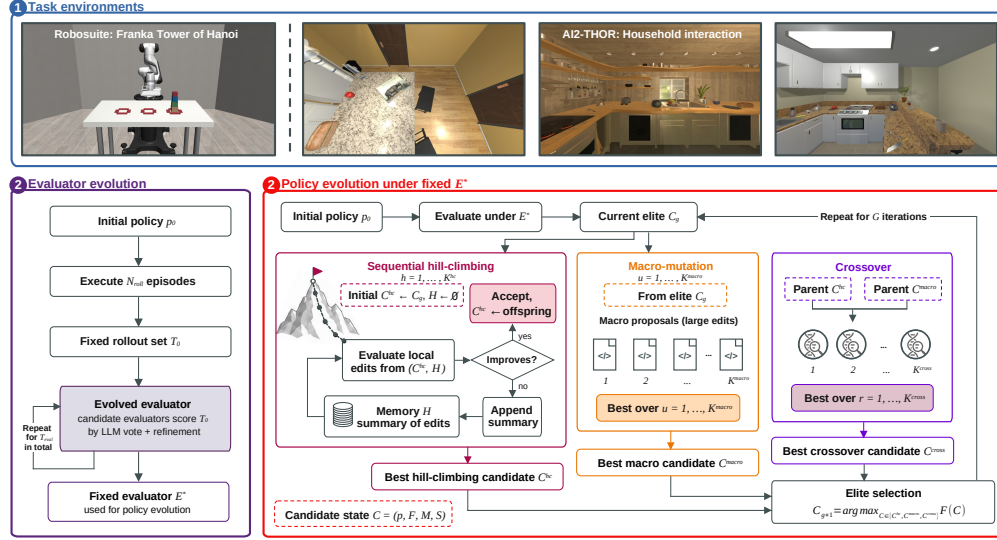


Figure 1: Conceptual overview of MEMENTO. The top row shows the task environments used in the study, including a Robosuite Franka Tower-of-Hanoi task and AI2-THOR household interaction scenes. The left panel illustrates evaluator evolution: candidate evaluators are generated, judged by an LLM majority vote, and iteratively refined to obtain the evolved evaluator E^* . The right panel illustrates policy evolution under the fixed evaluator E^* . An initial policy p_0 is evaluated to form the current elite candidate state C_g , after which each generation applies three proposal mechanisms: sequential hill-climbing from the current accepted candidate with rejected-proposal memory $\mathcal{H}$, macro-mutation from the elite, and crossover between the best hill-climb and macro-mutation candidates. The best branch output becomes the elite for the next generation. MEMENTO is evaluated on two long-horizon embodied domains: fully observed Robosuite Franka Tower-of-Hanoi (Zhu et al., 2020; Havur et al., 2013; Duggan et al., 2026), in which object placements define the legal Hanoi state for subsequent pick-and-place moves, and partially observed AI2-THOR household interaction (Kolve et al., 2017), in which valid

Local iterative improvement, search memory, and evolutionary variation via micro-mutation, macro-mutation, and crossover have not been combined in embodied policy programs that have been evaluated through environment rollouts. This limitation is addressed by MEMENTO, a memory-guided single-elite memetic search method for code-as-policy evolution in long-horizon embodied tasks. As illustrated in Figure 1, MEMENTO first evolves an evaluator E^* that maps policy rollouts to scalar fitness and structured feedback metrics and then freezes it for code-as-policy search. Starting from an executable policy program, candidates are generated in each generation through three revision branches: sequential hill-climbing through micro-mutations from the current accepted candidate with rejected-proposal memory, macro-mutation from the elite, and crossover between the best hill-climb and macro-mutation candidates. The best branch output becomes the elite for the next generation. MEMENTO is evaluated on two long-horizon embodied domains: fully observed Robosuite Franka Tower-of-Hanoi (Zhu et al., 2020; Havur et al., 2013; Duggan et al., 2026), in which object placements define the legal Hanoi state for subsequent pick-and-place moves, and partially observed AI2-THOR household interaction (Kolve et al., 2017), in which valid

object interactions depend on egocentric visibility and scene-specific object identifiers. The final Robosuite Tower-of-Hanoi policy is also deployed on a physical Franka robot, showcasing the feasibility of transferring policies evolved through simulation to real-world settings. In particular, the contributions are as follows:

- a single-elite memetic search framework for executable policy programs that combines sequential local improvement with rejected-proposal memory, macro-mutation, and crossover;
- an evaluator-evolution stage that produces a fixed evaluator before policy search, providing scalar fitness for selection and structured rollout feedback metrics for conditioning code-as-policy revisions;
- an empirical evaluation on two long-horizon embodied domains, including zero-shot, evaluator, and operator ablations, generalization across object-geometry and scene variations;
- a physical Franka deployment demonstrating sim-to-real transfer of the evolved Robosuite controller.

2 Related Work

Long-horizon embodied control. Long-horizon embodied tasks have been approached by constraining policy optimization with prior information. Extended manipulation behavior has been learned from teleoperated or language-annotated demonstrations in ACT (Zhao et al., 2023), Diffusion Policy (Chi et al., 2025), MimicPlay (Wang et al., 2023a), and BLADE (Liu et al., 2025), and demonstration and interaction data for compositional tasks are provided by the ALFRED (Shridhar et al., 2020), TEACH (Padmakumar et al., 2022), CALVIN (Mees et al., 2022), and BEHAVIOR (Srivastava et al., 2022) benchmarks. Task structure has also been specified through subtask sequences or program decompositions, as in Policy Sketches (Andreas et al., 2017) and Neural Task Programming (Xu et al., 2018), and ordering-constrained manipulation has been studied in rearrangement and Tower-of-Hanoi settings (Duggan et al., 2026; Kim et al., 2026). Sparse-reward reinforcement learning instead addresses delayed task feedback through goal relabeling, exploration, temporal abstraction, return decomposition, and model-based rollouts (Andrychowicz et al., 2017; Pathak et al., 2017; Ecoffet et al., 2021; Sutton et al., 1999; Arjona-Medina et al., 2019; Hafner et al., 2025). Across these approaches, delayed feedback is addressed by adding supervision, imposing temporal structure, or changing how experience is generated and credited during learning.

Language models for robot programs. Language models have been used to map task context to robot behavior, either by selecting among predefined skills or by generating executable programs. Action selection has been grounded with learned affordance and value estimates in SayCan (Ichter et al., 2023), and execution feedback and language-conditioned spatial reasoning have been incorporated into language-model planning in Inner Monologue (Huang et al., 2023b), VoxPoser (Huang et al., 2023a), and LLM-Planner (Song et al., 2023). Program-structured task plans have been produced from program-like prompts in ProgPrompt (Singh et al., 2023). Policy programs that process perception outputs and parameterize control-primitive APIs have been generated in Code as Policies (Liang et al., 2023), and in RoboPro (Xie et al., 2025) and GenCHiP (Burns et al., 2024) for visually grounded and contact-rich control. Rollout failures in generated policy programs have been characterized and used for failure-feedback

refinement in RoboInspector (Ying et al., 2025), and execution summaries have been used to explain failed robot trials and propose corrective actions in REFLECT (Liu et al., 2023). Together, these works show how language models can produce, ground, and refine robot programs from task context and execution feedback.

LLM-driven evolutionary search. This line of research follows genetic programming, in which executable programs are varied through mutation and crossover and selected for fitness (Koza, 1992). Language models have since been used as the variation operators, with model-conditioned edits replacing syntactic operators in ELM (Lehman et al., 2023) and LLM-GP (Hemberg et al., 2024). The methods that follow differ primarily in the object being searched. Discrete prompts have been evolved through model-generated mutation and crossover in EvoPrompt (Guo et al., 2023) and self-referentially in Promptbreeder (Fernando et al., 2023). Executable programs have been evolved by pairing a frozen model with a fixed evaluator in FunSearch (Romera-Paredes et al., 2024) and, at codebase scale, in AlphaEvolve (Novikov et al., 2025), and heuristics have been evolved through accumulated reflective feedback in ReEvo (Ye et al., 2024) and through memetic local search in EoH-S (Liu et al., 2026). Reward code has been evolved from model-generated variation selected by policy performance in Eureka (Ma et al., 2024), and with population-based crossover and human-feedback fitness in REvolve (Hazra et al., 2025). REvolve has also been extended from reward-function discovery to the discovery of executable reinforcement-learning algorithms, in which candidate update rules are evaluated through full training runs (Sygkounas et al., 2026b), while policy and environment programs are co-evolved in COvolve (Sygkounas et al., 2026a). Programmatic control policies have been evolved directly in MLES (Hu et al., 2026), where multimodal models generate candidate policy programs, selection uses quantitative task metrics, and new proposals are conditioned on behavioral evidence from rollouts. In these methods, the evaluator is fixed or externally specified, and the search object is a prompt, a heuristic, a reward, a program, or a program pair.

Memetic and memory-based search. Memetic algorithms combine population-level variation with local improvement of selected candidates (Moscato, 1989; Neri and Cotta, 2012). Recombination has been coupled with local refinement in crossover hill-climbing (Lozano et al., 2004), and local search has been concentrated on promising candidates through local-search chains (Molina et al., 2010). Memory-based meta-heuristics such as tabu search use search history to restrict recently explored moves and bias local improvement toward unexplored neighborhoods (Glover and Laguna, 1997). MEMENTO adapts this structure to model-generated code-as-policy: a single accepted candidate is refined by a memory-guided local-search branch that records rejected proposals, along with macro-mutation and crossover, all of which are selected by the evolved rollout evaluator.

3 Methodology

Sequential decision problems are modeled as finite-horizon POMDPs $\mathcal{M}_{\text{PO}} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, T, Z, R, \rho_0, L)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $\mathcal{O}$ the observation space, $T : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ the transition function, $Z : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{O})$ the observation function, $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ the environment reward, ρ_0 the initial-state distribution, and L the episode horizon (Kaelbling et al., 1998; Puterman, 1994). At time $t = 0, \dots, L - 1$, the environment has state $s_t \in \mathcal{S}$, the agent receives observation $o_t \in \mathcal{O}$, selects action $a_t \in \mathcal{A}$, and the environment transitions according to T . Task completion is represented by an episode-level success indicator $\sigma(\tau) \in \{0, 1\}$, defined on an executed trajectory τ .

The method does not directly optimize the environment reward R . Instead, it searches over executable code-as-policy programs. Each candidate program $p \in \mathcal{P}$ defines a deterministic stateful policy with internal program-state space $\mathcal{Z}_p$. At the start of each episode, the program initializes its internal state $z_0 \in \mathcal{Z}_p$. At timestep t , the executable program maps the current observation and internal program state to an action and an updated program state,

$$(a_t, z_{t+1}) = p(o_t, z_t), \quad o_t \in \mathcal{O}, a_t \in \mathcal{A}.$$

The induced policy is denoted by π_p . Executing π_p in $\mathcal{M}_{\text{PO}}$ for episode e induces a trajectory $\tau_e(p)$. For a rollout budget of N_{roll} episodes, let

$$\mathcal{T}_{N_{\text{roll}}}(p) = \{\tau_1(p), \dots, \tau_{N_{\text{roll}}}(p)\}$$

denote the resulting rollout set. A fixed evaluator E maps this rollout set to program-level fitness and feedback metrics,

$$E(\mathcal{T}_{N_{\text{roll}}}(p)) = (F(p), \mathbf{M}(p)).$$

Here $F(p) \in [0, 1]$ is the aggregate evaluator fitness over the N_{roll} episodes, and $\mathbf{M}(p)$ is the collection of rollout-level feedback metrics used to condition later code-as-policy revisions. The empirical success rate is computed from the task success indicator,

$$S(p) = \frac{1}{N_{\text{roll}}} \sum_{e=1}^{N_{\text{roll}}} \sigma(\tau_e(p)).$$

Evaluating a program defines the candidate state $\mathcal{C}(p) = (p, F(p), \mathbf{M}(p), S(p))$, which stores the program, the evaluator outputs, and the task-derived success rate. For any candidate state $\mathcal{C}$, $F(\mathcal{C})$ denotes the stored fitness value in $\mathcal{C}$.

3.1 Evaluator Evolution

Evaluator evolution searches for evaluator programs that map rollout data to scalar fitness and feedback metrics for code-as-policy search. The initial policy p_0 is executed for N_{roll} episodes, yielding the fixed rollout set $\mathcal{T}_0 = \mathcal{T}_{N_{\text{roll}}}(p_0)$ and empirical success rate $S_0 = S(p_0)$. The same rollout set is used for all evaluator candidates during evaluator evolution. At evaluator generation i , the search maintains a pool $\mathcal{E}_i = \{E_i^{(1)}, \dots, E_i^{(n)}\}$ of n evaluator programs. The initial pool $\mathcal{E}_0$ contains n LLM-generated evaluator programs. For $i > 0$, $\mathcal{E}_i$ is obtained by applying the evaluator macro-mutation operator $q_{\text{eval}}^{\text{macro}}$ to the selected evaluator from the previous generation, E_{i-1}^* .

Each evaluator candidate is applied to the shared rollout set and produces candidate-specific program-level outputs,

$$E_i^{(j)}(\mathcal{T}_0) = (F_i^{(j)}(p_0), \mathbf{M}_i^{(j)}(p_0)).$$

The empirical success rate S_0 is computed from the task success indicator and is shared across the evaluated pool. Let

$$\mathcal{D}_i = \left\{ (E_i^{(j)}, F_i^{(j)}(p_0), \mathbf{M}_i^{(j)}(p_0), S_0) \right\}_{j=1}^n$$

denote the scored evaluator pool. The selected evaluator is chosen by majority voting over k independent LLM judgments of $\mathcal{D}_i$,

$$E_i^* = \mathcal{V}_k(\mathcal{D}_i).$$

The voting step uses LLM-as-judge evaluation to compare the scored evaluator pool, aggregating by majority over k independent judgments (Zheng et al., 2023; Wang et al., 2023b). After T_{eval} evaluator generations, $E^* = E_{T_{\text{eval}}-1}^*$ is fixed and used for code-as-policy evolution. The full evaluator evolution process is outlined in Algorithm 1.

Algorithm 1 Evaluator Evolution

Require: initial policy p_0 , pool size n , rollout budget N_{roll} , voting budget k , evaluator generations T_{eval}

- 1: Execute p_0 for N_{roll} episodes; store $\mathcal{T}_0 = \mathcal{T}_{N_{\text{roll}}}(p_0)$ and $S_0 = S(p_0)$
- 2: Generate the initial evaluator pool $\{E_0^{(1)}, \dots, E_0^{(n)}\}$
- 3: **for** $i = 0$ to $T_{\text{eval}} - 1$ **do**
- 4: $\mathcal{D}_i \leftarrow \emptyset$
- 5: **for** $j = 1$ to n **do**
- 6: $(F_i^{(j)}, \mathbf{M}_i^{(j)}) \leftarrow E_i^{(j)}(\mathcal{T}_0)$
- 7: Add $(E_i^{(j)}, F_i^{(j)}, \mathbf{M}_i^{(j)}, S_0)$ to $\mathcal{D}_i$
- 8: **end for**
- 9: $E_i^* \leftarrow \mathcal{V}_k(\mathcal{D}_i)$ ▷ majority vote over k independent LLM judgments
- 10: $\{E_{i+1}^{(j)}\}_{j=1}^n \leftarrow \{q_{\text{eval}}^{\text{macro}}(E_i^*)\}_{j=1}^n$ ▷ unused when $i = T_{\text{eval}} - 1$
- 11: **end for**
- 12: **return** $E^* \leftarrow E_{T_{\text{eval}}-1}^*$

3.2 Policy Evolution Under the Fixed Evaluator

After E^* is fixed, policy search proceeds from a single elite candidate state $\mathcal{C}_g = \mathcal{C}(p_g)$. Each generation explores three branches: a hill-climb branch of sequential micro-mutations, a macro-mutation branch, and a crossover branch (cf. Figure 1). Let q_{hc} , q_{macro} , and q_{cross} denote LLM-conditioned proposal distributions that take evaluated candidate states as input and return new policy programs. Proposals are conditioned on the program text and the evaluator outputs F and $\mathbf{M}$ of the input candidate states; the success rate S is recorded for reporting and is not used during search. The complete policy evolution procedure is presented in Algorithm 2.

Hill-climb branch. For any proposed policy program p , let $\text{history}(p)$ denote the edit summary embedded in the program text, describing the change made relative to its parent. The hill-climb branch maintains a current accepted candidate state $\mathcal{C}_{g,h}^{\text{hc}} = \mathcal{C}(p_{g,h}^{\text{hc}})$ and a rejected-proposal memory $\mathcal{H}_{g,h}$, where $\mathcal{H}_{g,h}$ is an ordered list of edit summaries from rejected hill-climb proposals in generation g . Edit summaries of accepted proposals remain embedded in the accepted program text and are therefore visible to subsequent proposals through $\mathcal{C}_{g,h}^{\text{hc}}$; $\mathcal{H}_{g,h}$ records only rejected edits. The branch is initialized from the generation elite, with $\mathcal{C}_{g,0}^{\text{hc}} = \mathcal{C}_g$, and starts with an empty rejected-proposal memory, $\mathcal{H}_{g,0} = \emptyset$. At step $h = 1, \dots, K_{\text{hc}}$, a new policy program is sampled from the current accepted candidate state and the rejected-proposal memory:

$$p_{g,h}^{\text{new}} \sim q_{\text{hc}}(\mathcal{C}_{g,h-1}^{\text{hc}}, \mathcal{H}_{g,h-1}).$$

The proposed program is executed and evaluated under E^* , producing

$$\mathcal{C}_{g,h}^{\text{new}} = \mathcal{C}(p_{g,h}^{\text{new}}).$$

Algorithm 2 Single-Elite Memetic Code-as-Policy Evolution

Require: Initial policy p_0 ; evaluator E^* ; generations G ; budgets $N_{\text{roll}}, K_{\text{hc}}, K_{\text{macro}}, K_{\text{cross}}$

```

1:  $\mathcal{C}_0 \leftarrow \mathcal{C}(p_0)$  ▷ evaluate initial policy under  $E^*$ 
2: for  $g = 0$  to  $G - 1$  do
3:    $\mathcal{C}^{\text{hc}} \leftarrow \mathcal{C}_g, \mathcal{H} \leftarrow \emptyset$  ▷ hill-climb branch
4:   for  $h = 1$  to  $K_{\text{hc}}$  do
5:      $p \sim q_{\text{hc}}(\mathcal{C}^{\text{hc}}, \mathcal{H})$ 
6:      $\mathcal{C}^{\text{new}} \leftarrow \mathcal{C}(p)$  ▷ execute  $\pi_p$  for  $N_{\text{roll}}$  episodes, evaluate under  $E^*$ 
7:     if  $F(\mathcal{C}^{\text{new}}) \geq F(\mathcal{C}^{\text{hc}})$  then
8:        $\mathcal{C}^{\text{hc}} \leftarrow \mathcal{C}^{\text{new}}$ 
9:     else
10:       $\mathcal{H} \leftarrow \text{append}(\mathcal{H}, \text{history}(p))$ 
11:    end if
12:  end for
13:  for  $u = 1$  to  $K_{\text{macro}}$  do ▷ macro-mutation branch
14:     $p_u \sim q_{\text{macro}}(\mathcal{C}_g); \mathcal{C}_u^{\text{macro}} \leftarrow \mathcal{C}(p_u)$ 
15:  end for
16:   $\mathcal{C}^{\text{macro}} \leftarrow \arg \max_{\mathcal{C} \in \{\mathcal{C}_g, \mathcal{C}_1^{\text{macro}}, \dots, \mathcal{C}_{K_{\text{macro}}}^{\text{macro}}\}} F(\mathcal{C})$ 
17:  for  $r = 1$  to  $K_{\text{cross}}$  do ▷ crossover branch
18:     $p_r \sim q_{\text{cross}}(\mathcal{C}^{\text{hc}}, \mathcal{C}^{\text{macro}}); \mathcal{C}_r^{\text{cross}} \leftarrow \mathcal{C}(p_r)$ 
19:  end for
20:   $\mathcal{C}^{\text{cross}} \leftarrow \arg \max_{\mathcal{C} \in \{\mathcal{C}_1^{\text{cross}}, \dots, \mathcal{C}_{K_{\text{cross}}}^{\text{cross}}\}} F(\mathcal{C})$ 
21:   $\mathcal{C}_{g+1} \leftarrow \arg \max_{\mathcal{C} \in \{\mathcal{C}^{\text{hc}}, \mathcal{C}^{\text{macro}}, \mathcal{C}^{\text{cross}}\}} F(\mathcal{C})$  ▷ elitist selection
22: end for
23: return  $\mathcal{C}_G$ 

```

The proposal is accepted if its evaluated fitness is at least that of the current accepted program:

$$\mathcal{C}_{g,h}^{\text{hc}} = \begin{cases} \mathcal{C}_{g,h}^{\text{new}}, & \text{if } F(\mathcal{C}_{g,h}^{\text{new}}) \geq F(\mathcal{C}_{g,h-1}^{\text{hc}}), \\ \mathcal{C}_{g,h-1}^{\text{hc}}, & \text{otherwise.} \end{cases} \quad (1)$$

If the proposal is accepted, the rejected-proposal memory is unchanged: $\mathcal{H}_{g,h} = \mathcal{H}_{g,h-1}$. If the proposal is rejected, its edit summary is appended to the memory: $\mathcal{H}_{g,h} = \text{append}(\mathcal{H}_{g,h-1}, \text{history}(p_{g,h}^{\text{new}}))$. The branch output is the accepted candidate state after the final hill-climb step,

$$\mathcal{C}_g^{\text{hc}} = \mathcal{C}_{g,K_{\text{hc}}}^{\text{hc}}.$$

Macro-mutation branch. For $u = 1, \dots, K_{\text{macro}}$, an independent mutation of the generation elite is sampled and evaluated,

$$p_{g,u}^{\text{macro}} \sim q_{\text{macro}}(\mathcal{C}_g), \quad \mathcal{C}_{g,u}^{\text{macro}} = \mathcal{C}(p_{g,u}^{\text{macro}}).$$

The branch output is the candidate state with the highest fitness among the generation elite and the macro-mutation candidates:

$$\mathcal{C}_g^{\text{macro}} = \arg \max_{\mathcal{C} \in \{\mathcal{C}_g, \mathcal{C}_{g,1}^{\text{macro}}, \dots, \mathcal{C}_{g,K_{\text{macro}}}^{\text{macro}}\}} F(\mathcal{C}). \quad (2)$$

Crossover branch. For $r = 1, \dots, K_{\text{cross}}$, a recombination of the two branch outputs is sampled and evaluated,

$$p_{g,r}^{\text{cross}} \sim q_{\text{cross}}(\mathcal{C}_g^{\text{hc}}, \mathcal{C}_g^{\text{macro}}), \quad \mathcal{C}_{g,r}^{\text{cross}} = \mathcal{C}(p_{g,r}^{\text{cross}}).$$

The crossover proposal is conditioned on both selected candidate states, using their policy programs and evaluator outputs. The branch output is the evaluated crossover

candidate with the highest program fitness:

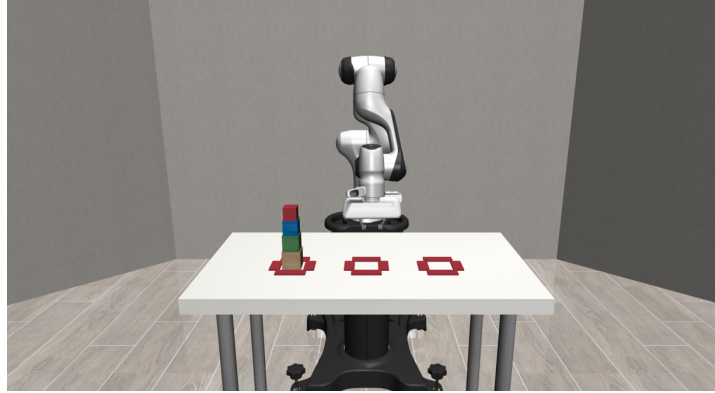
$$\mathcal{C}_g^{\text{cross}} = \arg \max_{\mathcal{C} \in \{\mathcal{C}_{g,1}^{\text{cross}}, \dots, \mathcal{C}_{g,K_{\text{cross}}}^{\text{cross}}\}} F(\mathcal{C}). \quad (3)$$

Generation elite selection. The branch output with the highest program fitness is selected as the next-generation elite and used as the parent for generation $g + 1$:

$$\mathcal{C}_{g+1} = \arg \max_{\mathcal{C} \in \{\mathcal{C}_g^{\text{hc}}, \mathcal{C}_g^{\text{macro}}, \mathcal{C}_g^{\text{cross}}\}} F(\mathcal{C}). \quad (4)$$

4 Experiments

We evaluate MEMENTO on two long-horizon embodied domains with different sources of execution dependence. In Robosuite Franka Tower-of-Hanoi (Zhu et al., 2020; Havur et al., 2013; Duggan et al., 2026), each pick-and-place operation changes the object configuration that determines the remaining legal moves. In AI2-THOR household interaction (Kolve et al., 2017), valid actions are constrained by egocentric visibility, scene-specific object identifiers, and object-interaction preconditions. The two domains, therefore, test code-as-policy search under structured manipulation state and partially observed household interaction. Figure 2 shows the training task settings used in the experiments.



(a) Robosuite Franka Tower-of-Hanoi



FloorPlan1



FloorPlan2



FloorPlan3

(b) AI2-THOR training kitchens

Figure 2: Training task environments. The Robosuite task is a four-object Tower-of-Hanoi manipulation problem in which the policy must move the full stack from the source container to the target container while preserving legal size order. The AI2-THOR task uses three training kitchens, FloorPlan1–FloorPlan3, which differ in layout, object placement, visibility, and receptacle configuration.

Robosuite Franka Tower-of-Hanoi. The Robosuite domain is a four-cube Tower-of-Hanoi task with a Franka Panda arm and three containers. At reset, all cubes are stacked on container A in legal size order. Success requires moving the complete tower to container C while leaving containers A and B empty. An episode terminates on success, timeout, or an invalid Hanoi placement in which a larger cube is placed on a smaller cube. For n cubes, the shortest legal solution requires $2^n - 1$ transfers; for $n = 4$, this gives 15 ordered pick-and-place operations.

AI2-THOR household interaction. The AI2-THOR domain uses the task `apple_in_microwave_on_bread_in_fridge`. Success requires placing the Apple in the Microwave, toggling the Microwave on, and placing the Bread in the Fridge. The agent receives egocentric observations, and object actions are valid only when the corresponding scene-specific object identifier appears in the current action space. An episode terminates on success, policy-issued `done`, or the step limit. Policy evolution uses `FloorPlan1–FloorPlan3`; cross-scene transfer is evaluated on held-out kitchens `FloorPlan4–FloorPlan29`.

The full observation and action interfaces are reported in Appendix A.2 and Appendix B; Robosuite domain-randomization details are reported in Appendix A.3. Held-out Robosuite object variants are shown in Appendix A.5, and additional AI2-THOR scene examples are shown in Appendix B.

Training setup. MEMENTO is run for $G = 5$ generations. Generation 0 denotes the evaluated initial policy p_0 and involves no policy search; each subsequent generation applies the three proposal branches. Each generation uses $K_{\text{hc}} = 10$ hill-climb proposals, $K_{\text{macro}} = 10$ macro-mutation proposals, and $K_{\text{cross}} = 4$ crossover proposals, giving a policy-candidate budget of $B = K_{\text{hc}} + K_{\text{macro}} + K_{\text{cross}} = 24$ new candidates per generation. Each candidate policy is evaluated for $N_{\text{roll}} = 10$ rollout episodes under the fixed evolved evaluator E^* . Evaluator evolution uses a pool of $n = 8$ evaluator candidates per generation for $T_{\text{eval}} = 5$ evaluator generations, with selection by majority vote over $k = 10$ independent LLM judgments. Results are reported as mean $\pm$ standard deviation across three random seeds. Curves report per-generation best-so-far fitness and empirical success rate, and tables report final-generation performance. Evaluator and code-as-policy proposals are generated with GPT-5.2 (OpenAI, 2025).

Baselines. Eureka (Ma et al., 2024) and REvolve (Hazra et al., 2025) are adapted from reward-code evolution to code-as-policy evolution: the search object is the executable policy program, and candidate fitness is computed by the same fixed evolved evaluator E^* used by MEMENTO. All methods start from the same seed-specific initial policy, use the same language model and domain prompt templates, and receive the same budget of $B = 24$ evaluated candidates per generation for $G = 5$ generations. The adapted Eureka samples independent macro-mutations of the current best program in each generation and selects the highest-fitness candidate. The adapted REvolve maintains a population of policy programs with fitness-based selection and LLM-generated crossover and macro-mutation, with E^* fitness replacing its human-feedback fitness.

5 Results

5.1 Main Evolution Results

We first evaluate whether the proposed search structure improves code-as-policy programs more effectively than LLM-guided evolutionary baselines under the same rollout budget. Figure 3 reports best-so-far evaluator fitness and empirical success rate

across generations for MEMENTO, REvolve, and Eureka. Results are reported as mean $\pm$ standard deviation across three random seeds, with all methods initialized from the same seed-specific policy at generation 0.

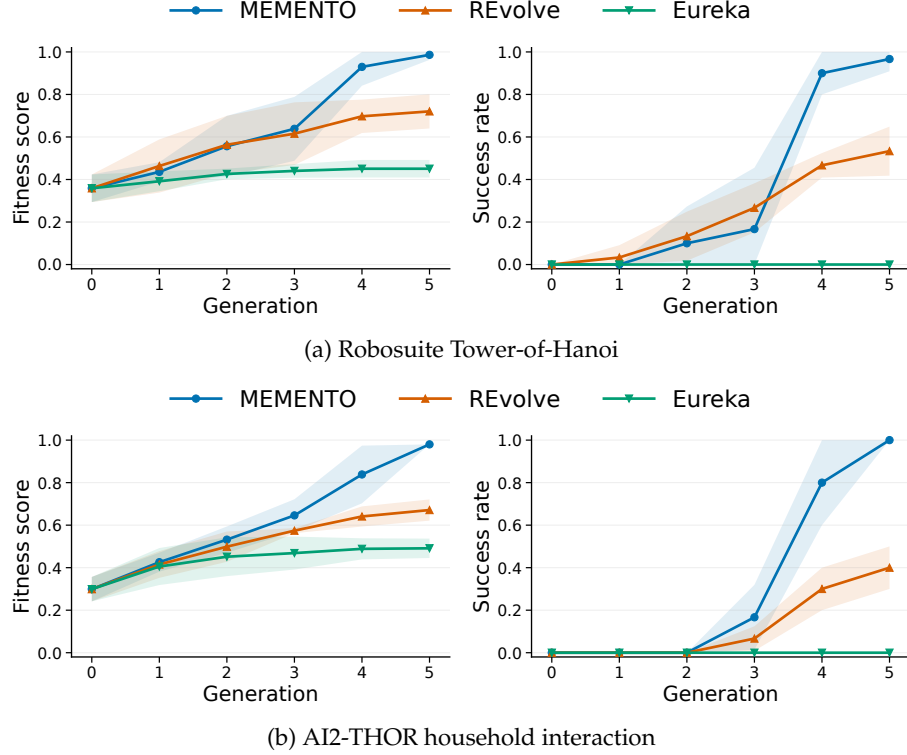


Figure 3: Main evolution results on Robosuite Tower-of-Hanoi and AI2-THOR household interaction. Subfigure (a) reports Robosuite and Subfigure (b) reports AI2-THOR. Within each subfigure, the left plot shows the best-so-far evaluator fitness, and the right plot shows the empirical success rate over generations. Lines denote the mean across three random seeds, and shaded regions denote one standard deviation for MEMENTO, REvolve, and Eureka.

By generation 5, MEMENTO achieves the strongest final performance in both domains. On Robosuite Tower-of-Hanoi, MEMENTO reaches 0.99 ± 0.02 fitness and 0.97 ± 0.06 success, compared with 0.72 ± 0.08 fitness and 0.53 ± 0.12 success for REvolve. Eureka reaches 0.45 ± 0.04 fitness and produces no successful rollouts. On AI2-THOR, MEMENTO reaches 0.98 ± 0.00 fitness and 1.00 ± 0.00 success, compared with 0.67 ± 0.05 fitness and 0.40 ± 0.10 success for REvolve; Eureka again produces no successful rollouts. These results show that, under the same rollout evaluator and candidate budget, MEMENTO’s memory-guided local refinement, macro-mutation, and crossover produce task-completing code-as-policy programs more reliably than the mutation-only and population-style evolutionary baselines.

Because candidate selection is performed using the fixed evaluator fitness F returned by E^* , while empirical success S is computed independently from the task-success predicate, we examined the correspondence between these two quantities. For each domain, the per-generation selected candidates, i.e., the elite $\mathcal{C}_g$ for MEMENTO

and the reported best candidate for each baseline, provide 54 paired values (F, S) , obtained from three methods, three seeds, and six generations. Fitness and success are positively correlated in both domains: Robosuite Tower-of-Hanoi gives Pearson $r = 0.922$ and Spearman $\rho = 0.755$, while AI2-THOR gives Pearson $r = 0.871$ and Spearman $\rho = 0.801$. These correlations indicate that the fixed evaluator provides a selection signal aligned with task success: candidates with higher evaluator fitness generally also achieve higher empirical success.

5.2 Final Performance and Generalization

Table 1 reports the success rate of the final selected policy after search on the original evaluation domains and on held-out generalization domains. For Robosuite Franka Tower-of-Hanoi, Cube denotes the cube-based environment used during policy evolution, while Asym. and Cyl. replace the manipulated cubes with held-out object geometries. These variants preserve the Hanoi objective, object identifiers, container layout, and structured observation interface, but change the grasp envelope and contact geometry of the manipulated objects; details and examples are given in Appendix A.5. For AI2-THOR, Search scenes denote the kitchens used during policy evolution, while Held-out scenes denote unseen kitchens with different layouts, object instances, visibility patterns, receptacle placements, and scene-specific object identifiers; environment details and scene examples are given in Appendix B. SAC+HER is included as a Robosuite-only interaction-learning baseline on the original cube task. Unlike the LLM-guided policy-code methods, SAC+HER is trained with goal-conditioned HER rewards rather than evaluator-conditioned code revision; its observation interface, training configuration, and sparse/dense reward definitions are reported in Appendices A.6–A.9.

Method	Robosuite Franka Tower-of-Hanoi			AI2-THOR household interaction	
	Cube	Asym.	Cyl.	Search scenes	Held-out scenes
Eureka	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
REvolve	0.53 ± 0.12	0.23 ± 0.15	0.00 ± 0.00	0.40 ± 0.10	0.08 ± 0.04
MEMENTO	0.97 ± 0.06	0.87 ± 0.23	0.63 ± 0.17	1.00 ± 0.00	0.78 ± 0.08
SAC+HER sparse	0.00 ± 0.00	–	–	–	–
SAC+HER dense	0.00 ± 0.00	–	–	–	–

Table 1: Final success rate on original and held-out generalization domains. Cube is the Robosuite Tower-of-Hanoi environment used during policy evolution. Asym. and Cyl. are held-out Robosuite object-geometry variants that preserve the Hanoi task while changing the shape of the manipulated object. Search scenes are the AI2-THOR kitchens used during policy evolution, and held-out scenes are unseen kitchens FloorPlan4--29. SAC+HER is evaluated only on the original Robosuite cube task. Entries are mean $\pm$ standard deviation across three seeds when available; “–” denotes not evaluated.

Table 1 shows that MEMENTO achieves the highest final success in both training domains and maintains substantial success under domain-specific generalization tests. In Robosuite, the final policy transfers from cubic training objects to asymmetric stackable objects and cylindrical disks, indicating that the evolved program relies on Hanoi state structure, legal stack ordering, and closed-loop placement geometry rather than memorizing cube-specific contacts. In AI2-THOR, the final policy transfers from the three search kitchens to unseen kitchens with new layouts, object placements, visibility

constraints, and object identifiers. REvolve partially solves the original domains, but loses most performance on the held-out Robosuite geometries and AI2-THOR kitchens, while Eureka produces no successful rollouts in either domain. These results indicate that the evolved programs rely on object-centric state assignments, legal subgoal ordering, and observation-conditioned action targets, with limited dependence on cube-specific contacts or memorized kitchen layouts.

5.3 Ablations

The ablations use the same three-seed evaluation protocol as the main comparison. All evolutionary code-as-policy search ablations use the same total candidate budget as the main experiments, $B = 24$ evaluated policy candidates per generation. The zero-shot no-evolution baseline uses the same one-generation candidate budget of 24 policy candidates, without evolutionary updates. We isolate four design choices: whether evolutionary code-as-policy search is needed at all, whether an evolved evaluator is needed to guide code-as-policy search, whether the code-as-policy search branches must be used jointly, and whether restricting AI2-THOR search to a single training scene reduces cross-scene generalization.

Zero-shot policy generation. We first test whether the LLM can solve the tasks without evolutionary policy search. In this setting, we sample 24 zero-shot policy candidates and evaluate the best candidate, but without evolutionary generations. This baseline tests whether the domains can be solved by single-round code generation alone.

On Robosuite Tower-of-Hanoi, zero-shot policy generation reaches 0.37 ± 0.12 fitness and 0.00 ± 0.00 success. On AI2-THOR, it reaches 0.22 ± 0.10 fitness and 0.00 ± 0.00 success. Thus, zero-shot policies achieve partial dense progress but never satisfy the full task-success predicate in either domain. By contrast, full MEMENTO reaches 0.99 ± 0.02 fitness and 0.97 ± 0.06 success on Robosuite, and 0.98 ± 0.00 fitness and 1.00 ± 0.00 success on AI2-THOR. Figure 4 shows this comparison. The zero-shot baseline is shown as a horizontal reference because it is not an evolutionary trajectory.

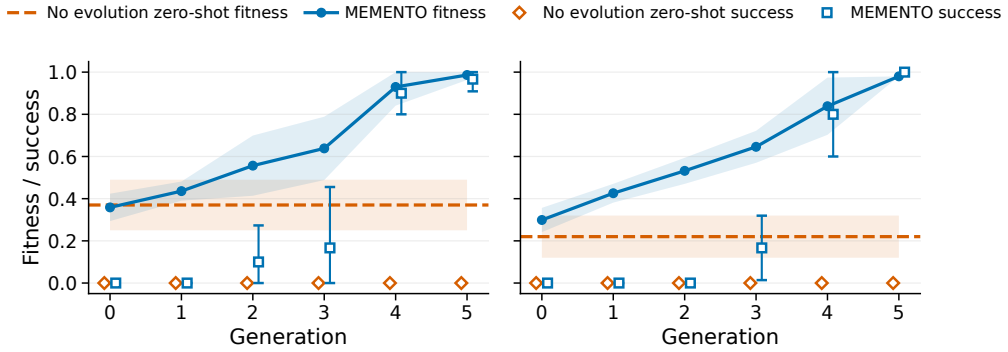


Figure 4: Zero-shot policy-generation baseline compared with full MEMENTO. Left: Robosuite Tower-of-Hanoi. Right: AI2-THOR household interaction. The dashed horizontal curve shows the no-evolution zero-shot fitness baseline from 24 policy candidates, and diamond markers show its empirical success. The solid curve shows MEMENTO best-so-far fitness, and square markers show MEMENTO empirical success. Shaded regions and error bars denote one standard deviation across three seeds.

Evaluator evolution. We next test whether evaluator evolution is needed once policy search is run. We replace the evolved evaluator from Section 3.1 with one zero-shot LLM-generated evaluator per seed and run the same policy-search procedure. The evaluator still returns scalar fitness and feedback metrics, but it is not mutated or selected through evaluator evolution. Figure 5 compares this no-evaluator-evolution variant with full MEMENTO.

At generation 5, the no-evaluator-evolution variant reaches 0.42 ± 0.05 fitness and 0.00 ± 0.00 success on Robosuite Tower-of-Hanoi, and 0.33 ± 0.05 fitness and 0.00 ± 0.00 success on AI2-THOR. Full MEMENTO reaches 0.99 ± 0.02 fitness and 0.97 ± 0.06 success on Robosuite, and 0.98 ± 0.00 fitness and 1.00 ± 0.00 success on AI2-THOR. The zero-shot evaluators support limited fitness improvement but do not guide policy search to task completion. This indicates that evaluator evolution is needed to align rollout feedback with the long-horizon success conditions.

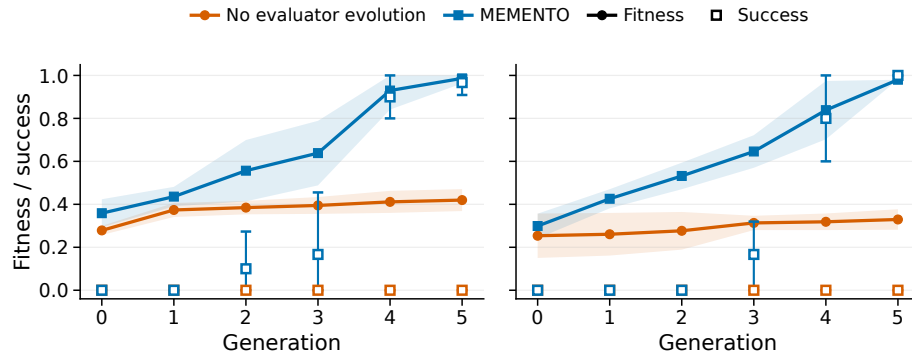


Figure 5: Evaluator-evolution ablation on Robosuite Tower-of-Hanoi and AI2-THOR household interaction. Each subfigure compares full MEMENTO with a variant that uses one zero-shot LLM-generated evaluator per seed and does not perform evaluator evolution. Solid curves show best-so-far evaluator fitness, and square markers show empirical success across generations. Lines and markers denote means across three seeds, and shaded regions denote one standard deviation.

Operator branches. We then evaluate whether the code-as-policy search branches must be used jointly. The evolved evaluator is kept fixed, and full MEMENTO is compared with two ablated variants: Hill-only and Without-Crossover. Hill-only reallocates the full candidate budget to sequential hill-climbing, using $K_{hc} = 24$, $K_{macro} = 0$, and $K_{cross} = 0$. Without-Crossover removes recombination and reallocates the crossover budget to macro-mutation, using $K_{hc} = 12$, $K_{macro} = 12$, and $K_{cross} = 0$. A macro-mutation-only variant is not included because it matches the mutation-only structure of the adapted Eureka baseline. Figure 6 reports the resulting fitness and success curves.

At generation 5, Hill-only reaches 0.50 ± 0.04 fitness on Robosuite Tower-of-Hanoi and 0.46 ± 0.04 on AI2-THOR, while Without-Crossover reaches 0.55 ± 0.02 and 0.51 ± 0.04 , respectively. Neither ablated variant produces successful rollouts in either domain. In contrast, full MEMENTO reaches 0.99 ± 0.02 fitness and 0.97 ± 0.06 success on Robosuite, and 0.98 ± 0.00 fitness and 1.00 ± 0.00 success on AI2-THOR. These results indicate that, under the tested candidate budget, local refinement and

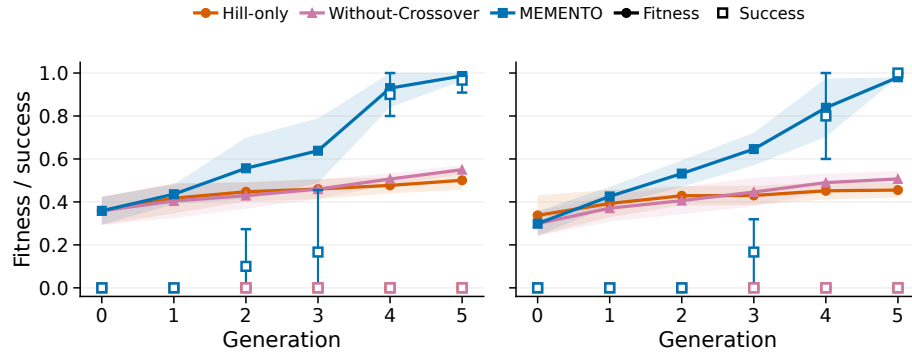


Figure 6: Operator-branch ablations on Robosuite Tower-of-Hanoi and AI2-THOR household interaction. Each subfigure compares Hill-only, Without-Crossover, and full MEMENTO. Solid curves show best-so-far evaluator fitness, and hollow square markers show empirical success across generations. Hill-only retains sequential accepted-candidate refinement with rejected-proposal memory but removes macro-mutation and crossover. Without-Crossover retains hill-climbing and macro-mutation but removes crossover between its best candidates.

macro-mutation alone are insufficient: the crossover branch is needed for MEMENTO to combine partial executable strategies into task-completing policies.

Search-scene diversity. Finally, we test whether AI2-THOR policy evolution requires multiple training kitchens. In this ablation, MEMENTO is restricted to a single training kitchen during policy evolution and is then evaluated across `FloorPlan1-FloorPlan29`. The resulting policies solve the search scene but generalize poorly across kitchens, reaching 0.09 ± 0.02 success across seeds (2/29, 3/29, 3/29). This suggests that single-scene policy evolution overfits to scene-specific object placements, navigation routes, and interaction viewpoints. In contrast, multi-scene policy evolution exposes the policy search to scene-dependent object identifiers, visibility constraints, receptacle locations, and valid-action preconditions. Together with the held-out scene results in Table 1, this shows that search-scene diversity is necessary for AI2-THOR cross-scene generalization.

5.4 Sim-to-Real Transfer

The physical deployment tests whether the evolved program logic completes the task when its object-centric inputs are estimated from real RGB-D perception, and its selected moves are executed through physical contact. The best-performing Robosuite Tower-of-Hanoi code-as-policy across the three MEMENTO seeds, denoted p_{best} , was deployed on a physical Franka robot. The evolved task-level code is kept fixed. During the simulation search, the selected policy acquired a closed-loop execution structure that reduces dependence on exact simulator dynamics: object poses are filtered before target computation, pick-and-place targets are re-derived from the current cube and container estimates, and contact-sensitive execution is guarded by alignment checks, placement centering, grasp verification, and post-placement retry. The symbolic Hanoi ordering remains fixed, while the execution of each scheduled move is adapted to the currently estimated configuration of objects. Additional details of the evolved Hanoi

policy are given in Appendix C.2.1.

Simulator object states, and end-effector control increments are replaced by RGB-D object-pose estimates and Franka motion commands. During the simulation search, observation noise and calibration bias were sampled from the randomization model in Appendix A.3. In the physical trials, analogous errors arise from segmentation, depth measurement, camera calibration, and contact execution. In simulation, Robosuite provides object poses and robot state, and actions are applied through the Robosuite end-effector controller. In the physical setup, object and container poses are estimated from RGB-D observations using SAM3 segmentation (Carion et al., 2026), and policy-selected pick-and-place targets are executed by the robot motion planner and gripper controller.

Before the final robot evaluation, a small set of execution constants was recalibrated for the physical cell over $N_{\text{cal}} = 3$ LLM-assisted calibration steps. The calibration variables, real-robot metrics, and calibration procedure are detailed in Appendix A.4. The real-robot metrics correspond to a subset of the evaluator’s feedback categories and are computed by a reduced evaluator E_R . They guide calibration only; no fitness is computed, and no policy search is performed during deployment.

The calibrated code-as-policy achieved a success rate of 0.90 over $N_{\text{real}} = 10$ physical trials. In all 10 trials, the task-level code followed the fixed Hanoi ordering and rederived execution targets from the estimated object configuration; the single unsuccessful trial was due to a perception error.

6 Limitations and Future Work

The experiments use two task formulations: four-cube Tower-of-Hanoi manipulation in Robosuite and a long-horizon household-interaction task in AI2-THOR. Since each evolved program is conditioned on a task-specific prompt, evaluator, and success predicate, broader task coverage would require running the full evaluator-evolution and code-as-policy search procedure on additional task definitions. Model sensitivity is not isolated, as evaluator and policy proposals are generated using a single language model. The evaluator E^* is evolved from the initial-policy rollout set $\mathcal{T}_0$ and then fixed during code-as-policy search, so the experiments do not test whether evaluator evolution should be reintroduced when later policy elites expose failures absent in the initial rollouts. The hill-climb branch is sequential within each generation because each proposal depends on the currently accepted candidate and the rejection memory, thereby limiting within-branch parallelism relative to population-based variation.

Future work should therefore extend the current single-elite formulation to a population-based code-as-policy search. Multiple islands could maintain separate policy elites, run branch-level search independently, and periodically exchange high-fitness candidates through migration (Cantú-Paz et al., 1998). This would add population diversity and reduce the sequential bottleneck of a single hill-climb chain. In addition, future work should also study adaptive evaluator–policy co-evolution, in which E^* is periodically re-evolved from rollouts of later elites or from failed and ambiguous candidates collected during the search, rather than only from p_0 . Finally, evaluator and policy generation with multiple language models should also be tested to measure model sensitivity.

7 Conclusion

This work introduced MEMENTO, a memory-guided single-elite memetic framework for evolving executable code-as-policy programs in long-horizon embodied tasks. The

method first evolves a rollout evaluator that maps executions to scalar fitness and task-specific feedback metrics, then uses the fixed evaluator to guide policy search through sequential hill-climbing with rejected-proposal memory, macro-mutation, and crossover between the best hill-climb and macro-mutation candidates. On the Robosuite Franka Tower-of-Hanoi and AI2-THOR household interaction tasks, MEMENTO achieved higher performance than LLM-guided evolutionary baselines adapted to code-as-policy search, including on held-out object geometries and unseen household scenes. Ablations showed that zero-shot policy generation, policy search with an unevolved zero-shot evaluator, and removal of search branches each reduced final performance. Finally, the best evolved Robosuite Tower-of-Hanoi code-as-policy was deployed on a physical Franka robot. These results support memory-guided memetic search as a mechanism to improve executable embodied policies using sparse rollout feedback.

References

- Andreas, J., Klein, D., and Levine, S. (2017). Modular multitask reinforcement learning with policy sketches. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 166–175. PMLR.
- Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, P., and Zaremba, W. (2017). Hindsight experience replay. In *Advances in Neural Information Processing Systems*, volume 30.
- Angeline, P. J. (1997). Subtree crossover: Building block engine or macromutation? In *Genetic Programming 1997: Proceedings of the Second Annual Conference*, pages 9–17. Morgan Kaufmann.
- Arjona-Medina, J. A., Gillhofer, M., Widrich, M., Unterthiner, T., Brandstetter, J., and Hochreiter, S. (2019). RUDDER: Return decomposition for delayed rewards. In *Advances in Neural Information Processing Systems*, volume 32, pages 13544–13555.
- Brameier, M. F. and Banzhaf, W. (2007). *Linear Genetic Programming*. Springer.
- Burns, K., Jain, A., Go, K., Xia, F., Stark, M., Schaal, S., and Hausman, K. (2024). GenCHiP: Generating robot policy code for high-precision and contact-rich manipulation tasks. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9596–9603. IEEE.
- Cantú-Paz, E. et al. (1998). A survey of parallel genetic algorithms. *Calculateurs paralleles, reseaux et systems repartis*, 10(2):141–171.
- Carion, N., Gustafson, L., Hu, Y.-T., Debnath, S., Hu, R., Suris, D., Ryali, C., Alwala, K. V., Khedr, H., Huang, A., Lei, J., Ma, T., Guo, B., Kalla, A., Marks, M., Greer, J., Wang, M., Sun, P., Rädle, R., Afouras, T., Mavroudi, E., Xu, K., Wu, T.-H., Zhou, Y., Momeni, L., Hazra, R., Ding, S., Vaze, S., Porcher, F., Li, F., Li, S., Kamath, A., Cheng, H. K., Dollár, P., Ravi, N., Saenko, K., Zhang, P., and Feichtenhofer, C. (2026). SAM 3: Segment anything with concepts. In *International Conference on Learning Representations*.
- Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. (2025). Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10–11):1684–1704.
- Duggan, T., Lorang, P., Lu, H., and Scheutz, M. (2026). The price is not right: Neuro-symbolic methods outperform vllms on structured long-horizon manipulation tasks with significantly lower energy consumption. *arXiv preprint arXiv:2602.19260*.
- Echevarrieta, J., Arza, E., and Pérez, A. (2025). Speeding-up evolutionary algorithms to solve black-box optimization problems. *IEEE Transactions on Evolutionary Computation*, 29(1):117–131.

- Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O., and Clune, J. (2021). Go-explore: A new approach for hard-exploration problems. *Artificial Intelligence*, 297:103503.
- Fernando, C., Banarse, D., Michalewski, H., Osindero, S., and Rocktäschel, T. (2023). Prompt-breeder: Self-referential self-improvement via prompt evolution.
- Glover, F. and Laguna, M. (1997). *Tabu Search*. Springer, New York.
- Guo, Q., Wang, R., Guo, J., Li, B., Song, K., Tan, X., Liu, G., Bian, J., and Yang, Y. (2023). Evo-Prompt: Connecting large language models with evolutionary algorithms yields powerful prompt optimizers.
- Hafner, D., Pasukonis, J., Ba, J., and Lillicrap, T. (2025). Mastering diverse control tasks through world models. *Nature*, 640:647–653.
- Havur, G., Haspalamutgil, K., Palaz, C., Erdem, E., and Patoglu, V. (2013). A case study on the tower of hanoi challenge: Representation, reasoning and execution. In *Proceedings of the IEEE International Conference on Robotics and Automation*, pages 4552–4559.
- Hazra, R., Sygkounas, A., Persson, A., Loutfi, A., and Dos Martires, P. Z. (2025). REvolve: Reward evolution with large language models using human feedback. In *International Conference on Learning Representations*.
- Hemberg, E., Moskal, S., and O’Reilly, U.-M. (2024). Evolving code with a large language model. *Genetic Programming and Evolvable Machines*, 25(2):21.
- Hu, Q., Tong, X., Yuan, M., Liu, F., Lu, Z., and Zhang, Q. (2026). Multimodal LLM-assisted evolutionary search for programmatic control policies. In *International Conference on Learning Representations*.
- Huang, W., Wang, C., Zhang, R., Li, Y., Wu, J., and Fei-Fei, L. (2023a). VoxPoser: Composable 3d value maps for robotic manipulation with language models. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 540–562. PMLR.
- Huang, W., Xia, F., Xiao, T., Chan, H., Liang, J., Florence, P., Zeng, A., Tompson, J., Mordatch, I., Chebotar, Y., Sermanet, P., Jackson, N., Brown, N., Luu, L., Levine, S., Hausman, K., and Ichter, B. (2023b). Inner monologue: Embodied reasoning through planning with language models. In *Proceedings of the 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 1769–1782. PMLR.
- Ichter, B., Brohan, A., Chebotar, Y., Finn, C., Hausman, K., Herzog, A., Ho, D., Ibarz, J., Irpan, A., Jang, E., Julian, R., Kalashnikov, D., Levine, S., Lu, Y., Parada, C., Rao, K., Sermanet, P., Toshev, A. T., Vanhoucke, V., Xia, F., Xiao, T., Xu, P., Yan, M., Brown, N., Ahn, M., Cortes, O., Sievers, N., Tan, C., Xu, S., Reyes, D., Rettinghouse, J., Quiambao, J., Pastor, P., Luu, L., Lee, K.-H., Kuang, Y., Jesmonth, S., Joshi, N. J., Jeffrey, K., Jauregui Ruano, R., Hsu, J., Gopalakrishnan, K., David, B., Zeng, A., and Fu, C. (2023). Do as i can, not as i say: Grounding language in robotic affordances. In *Proceedings of the 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR.
- Kaelbling, L. P., Littman, M. L., and Cassandra, A. R. (1998). Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134.
- Kim, M., Chen, K.-C., Shrestha, A., Fuxin, L., Lee, S., and Fern, A. (2026). Humanoid hanoi: Investigating shared whole-body control for skill-based box rearrangement.
- Kolve, E., Mottaghi, R., Han, W., VanderBilt, E., Weihs, L., Herrasti, A., Deitke, M., Ehsani, K., Gordon, D., Zhu, Y., Kembhavi, A., Gupta, A., and Farhadi, A. (2017). AI2-THOR: An interactive 3d environment for visual ai.
- Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA, USA.

- Lehman, J., Gordon, J., Jain, S., Ndousse, K., Yeh, C., and Stanley, K. O. (2023). Evolution through large models. In *Handbook of Evolutionary Machine Learning*, pages 331–366. Springer.
- Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., and Zeng, A. (2023). Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500. IEEE.
- Liu, F., Liu, Y., Zhang, Q., Tong, X., and Yuan, M. (2026). EoH-S: Evolution of heuristic set using LLMs for automated heuristic design. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 37090–37098.
- Liu, S., Yuan, H., Hu, M., Li, Y., Chen, Y., Liu, S., Lu, Z., and Jia, J. (2024). RL-GPT: Integrating reinforcement learning and code-as-policy. In *Advances in Neural Information Processing Systems*, volume 37, pages 28430–28459.
- Liu, W., Nie, N., Zhang, R., Mao, J., and Wu, J. (2025). Learning compositional behaviors from demonstration and language. In *Proceedings of the 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 1992–2028. PMLR.
- Liu, Z., Bahety, A., and Song, S. (2023). REFLECT: Summarizing robot experiences for failure explanation and correction. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 3468–3484. PMLR.
- Lozano, M., Herrera, F., Krasnogor, N., and Molina, D. (2004). Real-coded memetic algorithms with crossover hill-climbing. *Evolutionary Computation*, 12(3):273–302.
- Ma, Y. J., Liang, W., Wang, G., Huang, D.-A., Bastani, O., Jayaraman, D., Zhu, Y., Fan, L., and Anandkumar, A. (2024). Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations*.
- Mees, O., Hermann, L., Rosete-Beas, E., and Burgard, W. (2022). CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334.
- Molina, D., Lozano, M., García-Martínez, C., and Herrera, F. (2010). Memetic algorithms for continuous optimisation based on local search chains. *Evolutionary Computation*, 18(1):27–63.
- Moscato, P. (1989). On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms. *Caltech Concurrent Computation Program, C3P Report*, 826:37.
- Neri, F. and Cotta, C. (2012). Memetic algorithms and memetic computing optimization: A literature review. *Swarm and Evolutionary Computation*, 2:1–14.
- Novikov, A., Vū, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J. R., Mehrabian, A., Kumar, M. P., See, A., Chaudhuri, S., Holland, G., Davies, A., Nowozin, S., Kohli, P., and Balog, M. (2025). AlphaEvolve: A coding agent for scientific and algorithmic discovery. Technical report, Google DeepMind. Technical report, arXiv:2506.13131.
- OpenAI (2025). Introducing gpt-5.2. <https://openai.com/index/introducing-gpt-5-2/>. Accessed: 2026-06-09.
- Padmakumar, A., Thomason, J., Shrivastava, A., Lange, P., Narayan-Chen, A., Gella, S., Piramuthu, R., Tur, G., and Hakkani-Tur, D. (2022). TEACH: Task-driven embodied agents that chat. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 2017–2025.
- Pathak, D., Agrawal, P., Efros, A. A., and Darrell, T. (2017). Curiosity-driven exploration by self-supervised prediction. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*.
- Puterman, M. L. (1994). *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley Series in Probability and Statistics. John Wiley & Sons.

- Romera-Paredes, B., Barekatin, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J. R., Ellenberg, J. S., Wang, P., Fawzi, O., Kohli, P., and Fawzi, A. (2024). Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475.
- Shridhar, M., Thomason, J., Gordon, D., Bisk, Y., Han, W., Mottaghi, R., Zettlemoyer, L., and Fox, D. (2020). ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10740–10749.
- Singh, I., Blukis, V., Mousavian, A., Goyal, A., Xu, D., Tremblay, J., Fox, D., Thomason, J., and Garg, A. (2023). ProgPrompt: Program generation for situated robot task planning using large language models. *Autonomous Robots*, 47:999–1012.
- Song, C. H., Wu, J., Washington, C., Sadler, B. M., Chao, W.-L., and Su, Y. (2023). LLM-Planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2998–3009.
- Srivastava, S., Li, C., Lingelbach, M., Martín-Martín, R., Xia, F., Vainio, K. E., Lian, Z., Gokmen, C., Buch, S., Liu, K., Savarese, S., Gweon, H., Wu, J., and Fei-Fei, L. (2022). BEHAVIOR: Benchmark for everyday household activities in virtual, interactive, and ecological environments. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 477–490. PMLR.
- Sutton, R. S., Precup, D., and Singh, S. (1999). Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211.
- Sygkounas, A., Hazra, R., Persson, A., Martires, P. Z. D., and Loutfi, A. (2026a). COvolve: Adversarial co-evolution of large-language-model-generated policies and environments via two-player zero-sum game. In *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO '26*, New York, NY, USA. Association for Computing Machinery.
- Sygkounas, A., Loutfi, A., and Persson, A. (2026b). Evolutionary discovery of reinforcement learning algorithms via large language models. In *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO '26*, New York, NY, USA. Association for Computing Machinery.
- Wang, C., Fan, L., Sun, J., Zhang, R., Fei-Fei, L., Xu, D., Zhu, Y., and Anandkumar, A. (2023a). Mimicplay: Long-horizon imitation learning by watching human play. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 201–221. PMLR.
- Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. (2023b). Self-consistency improves chain of thought reasoning in language models. In *Proc. Int. Conf. Learn. Represent. (ICLR)*.
- Xie, S., Wang, H., Xiao, Z., Wang, R., and Chen, X. (2025). Robotic programmer: Video instructed policy code generation for robotic manipulation. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 14923–14930. IEEE.
- Xu, D., Nair, S., Zhu, Y., Gao, J., Garg, A., Fei-Fei, L., and Savarese, S. (2018). Neural task programming: Learning to generalize across hierarchical tasks. In *2018 IEEE international conference on robotics and automation (ICRA)*, pages 3795–3802. IEEE.
- Ye, H., Wang, J., Cao, Z., Berto, F., Hua, C., Kim, H., Park, J., and Song, G. (2024). ReEvo: Large language models as hyper-heuristics with reflective evolution. *Advances in Neural Information Processing Systems*, 37:43571–43608.
- Ying, C., Du, L., Cheng, P., and Shu, Y. (2025). RoboInspector: Unveiling the unreliability of policy code for LLM-enabled robotic manipulation.
- Zhao, T. Z., Kumar, V., Levine, S., and Finn, C. (2023). Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea.

- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36.
- Zhu, Y., Wong, J., Mandlekar, A., Martín-Martín, R., Joshi, A., Lin, K., Maddukuri, A., Nasiriany, S., and Zhu, Y. (2020). robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*.

The appendix is organized as follows. Appendix A details the Robosuite Franka Tower-of-Hanoi environment, domain randomization, the physical Franka deployment, held-out object variants, and the SAC+HER baselines. Appendix B details the AI2-THOR environment, the code-as-policy interface, and the observation space. Appendix C reports the evolved evaluators and representative evolved policy programs.

A Robosuite Franka Tower-of-Hanoi Environment

A.1 Task and Simulator Setup

The Robosuite task uses a Franka Panda arm, four cubes, and three containers. The cubes are ordered by size: extra-large, large, medium, and small. At reset, all four cubes are stacked on container A in legal Tower-of-Hanoi order, with the extra-large cube at the bottom and the small cube at the top. The goal is to move the complete tower to container C, preserve the same size order, and leave containers A and B empty. An episode terminates on success, illegal Hanoi placement, or timeout after 12000 environment steps.

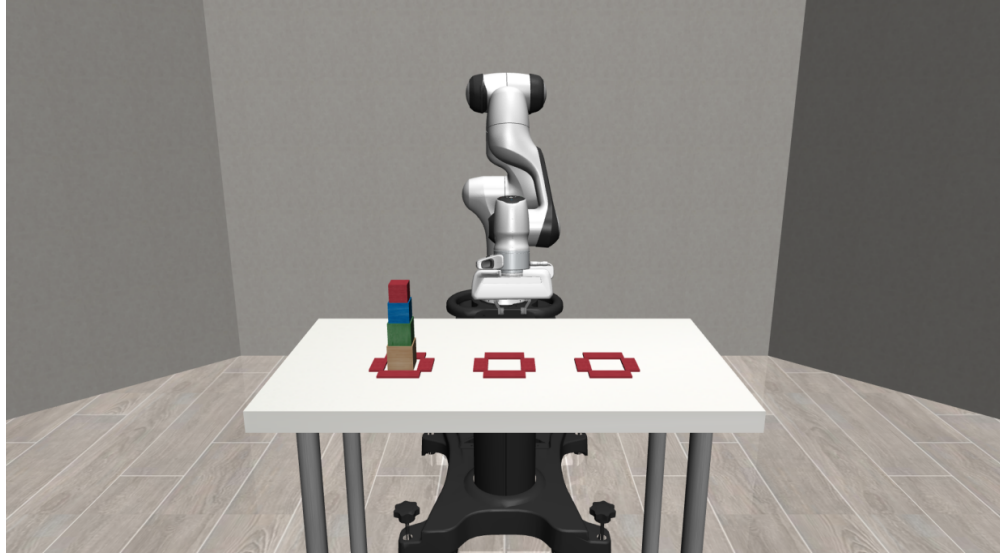


Figure 7: Initial reset state of the Robosuite Franka Panda Tower-of-Hanoi task. Four cubes are stacked on container A; the target is to move the full tower to container C while preserving the size order.

The cube half-edge lengths are fixed by the environment:

$$s_{\text{small}} = 0.022, \quad s_{\text{medium}} = 0.025, \quad s_{\text{large}} = 0.028, \quad s_{\text{xlarge}} = 0.031.$$

The containers are plate-with-hole objects. The hole is shifted from the container origin along the container-local positive x -axis. The local hole interval is

$$x_{\text{local}} \in [0.02, 0.09], \quad |y_{\text{local}}| \leq 0.035,$$

so the nominal hole center is at local offset

$$h_{\text{local}} = [0.055, 0, 0]^{\top}.$$

For a container with world position b and orientation R_b , the placement center used by policy code is

$$h(b, R_b) = b + R_b h_{\text{local}}.$$

A.2 Observation and Action Space

Policy-code evolution generates a Python class `FrankaPolicy` with two required methods. The method `reset()` clears episode-level policy state before each rollout. The method `compute_action(obs)` is called at every control step and returns a continuous seven-dimensional action vector:

$$a_t = [\Delta x, \Delta y, \Delta z, \Delta \phi, \Delta \theta, \Delta \psi, \Delta g] \in [-1, 1]^7.$$

The first three entries are end-effector position increments, the next three entries are axis-angle orientation increments, and the final entry controls the gripper. Positive gripper values close the gripper, negative values open it, and zero holds the current command.

The generated policy receives the structured Robosuite observation dictionary directly. The task-relevant observation fields are listed in Table 2. The policy is instructed to use these structured keys rather than packed aggregate vectors.

Table 2: Observation fields used by policy-code evolution in the Robosuite Franka Tower-of-Hanoi task.

Group	Key	Content
Robot	<code>robot0_eef_pos</code>	End-effector position, shape (3,)
Robot	<code>robot0_eef_quat</code>	End-effector quaternion, shape (4,)
Robot	<code>robot0_gripper_qpos</code>	Gripper joint positions, shape (2,)
Robot	<code>robot0_gripper_qvel</code>	Gripper joint velocities, shape (2,)
Robot	<code>robot0_joint_pos</code>	Arm joint positions, shape (7,)
Robot	<code>robot0_joint_vel</code>	Arm joint velocities, shape (7,)
Cubes	<code>cube_pos</code>	Four cube positions, shape (12,)
Cubes	<code>cube_quat</code>	Four cube quaternions, shape (16,)
Cubes	<code>cube_size</code>	Four cube half-edge lengths, shape (4,)
Containers	<code>boxes_pos</code>	Container A/B/C positions, shape (9,)
Containers	<code>boxes_quat</code>	Container A/B/C quaternions, shape (12,)

The evolved policy parses the four cube poses and sizes, computes the hole center of each container from `boxes_pos` and `boxes_quat`, and infers the current cube-to-container assignment. Its symbolic component generates the recursive Tower-of-Hanoi move sequence from container *A* to container *C* through container *B*, yielding $2^4 - 1 = 15$ ordered moves. For each scheduled move, the policy selects the top observed cube on the source container, recomputes pick-and-place targets from the current object and container poses, and executes the move via closed-loop pick-and-place control. The symbolic Hanoi ordering remains fixed; closed-loop adaptation comes from observation-based cube assignment, grasp/lift checks, and geometric target recomputation during execution.

A.3 Domain Randomization for Policy-Code Evolution

At each reset, the environment randomizes observation noise, calibration bias, contact parameters, cube mass, joint damping, actuator gain, world position offset, and initial cube placement. These perturbations modify both the object-state observations received by the policy and the underlying MuJoCo dynamics, while preserving the same task definition and success condition.

The position-noise standard deviation is sampled once per episode and then used when cube and container positions are observed. The resulting sensor value is the true object position plus an episode-level position bias and per-observation Gaussian noise. The world XY offset changes the physical placement of containers A, B, and C at reset. The initial cube XY perturbation applies an independent physical offset to each cube in the initial tower. Observation latency is not included in Table 3, because it is not applied to the returned observations in the implementation used for the experiments.

A.4 Physical Franka Deployment

This appendix details the physical Franka deployment used for the sim-to-real result in Section 5.4. The deployed policy is the best-performing Robosuite Tower-of-Hanoi code-as-policy

Table 3: Heavy domain randomization used for policy-code evolution in the Robosuite Franka Tower-of-Hanoi environment.

Quantity	Value
Sliding friction scale	[0.3, 1.8]
Torsional friction	[0.001, 0.01]
Rolling friction	[0.0001, 0.001]
Position-noise standard deviation	[0.002, 0.025]
Position bias	± 0.015
World XY offset	± 0.015
Cube mass scale	[0.7, 1.5]
Joint damping scale	[0.8, 1.5]
Actuator gain scale	[0.85, 1.15]
Initial cube XY perturbation	± 0.003

selected across the MEMENTO seeds. The selected policy uses a fixed, recursive Tower-of-Hanoi move schedule, together with guarded, closed-loop execution of each pick-and-place move. The evolved program logic is kept fixed, while the simulator observation and action interfaces are replaced by real-robot perception and execution.

Deployment interface. Let p_{best} denote the selected Robosuite code-as-policy. In simulation, p_{best} receives the Robosuite observation dictionary o_t , including robot state, cube poses, cube sizes, and container poses, and returns a seven-dimensional end-effector action. On the physical robot, a deployment wrapper converts RGB-D observations and robot-state measurements into the same object-centric fields consumed by the policy:

$$o_t^R = \Psi_R(y_t^R),$$

where y_t^R denotes the raw real-robot observation and Ψ_R estimates the cube poses, cube sizes, container poses, and end-effector pose in the Franka base frame.

Given o_t^R , the physical execution version of the policy evaluates the same internal finite-state controller as in simulation. At each control step, the policy identifies the active manipulation phase, computes the current Cartesian pick or place target, and issues the corresponding gripper command:

$$(\alpha_t, x_t^*, g_t, z_{t+1}) = p_{\text{best}}^R(o_t^R, z_t).$$

Here α_t is the active manipulation phase, x_t^* is the Cartesian target, g_t is the gripper command, and z_t is the policy state. The physical execution stack maps these phase-specific targets to Franka motion and gripper commands,

$$u_t^R = \Gamma_R(\alpha_t, x_t^*, g_t).$$

Thus, Ψ_R replaces the simulator observation source and Γ_R replaces the simulator action interface, while the Tower-of-Hanoi state machine remains fixed.

Preserved program logic. The preserved policy components are the object-centric parts of the evolved code-as-policy. These include the recursive 15-move Tower-of-Hanoi schedule, the active move index, the manipulation finite-state machine, filtered pose estimates, retry-conditioned pick offsets, guarded descent, stack-height-based placement-target computation, grasp verification, placement centering, and post-placement verification. The deployed policy does not perform global BFS replanning. Instead, the symbolic Hanoi ordering remains fixed, and recovery is local to the active move: failed grasps or failed post-placement checks cause the same scheduled move to be retried from the updated object observation.

RGB-D object grounding. Robosuite object-state observations are replaced by RGB-D object grounding. SAM3-based language-guided segmentation is queried with the physical object labels used in the task: the four cube colors and the three container markers (Carion et al., 2026). The resulting segmentation masks are combined with depth measurements and calibrated camera extrinsics to estimate object poses in the Franka base frame. The estimated cube poses, cube

sizes, container poses, and end-effector pose are written into the same object-centric fields consumed by p_{best} . Cube poses are estimated online during execution. Container poses are measured once and reused as fixed container locations during the physical trials.

Physical execution. Policy-selected pick-and-place targets are executed by the Franka motion and gripper stack. Long transfers between task locations are executed through motion planning. Short vertical motions for grasping, lifting, placing, and retreating are executed as local Cartesian motions. The gripper command is provided by the policy and executed by the physical wrapper. After each executed phase, the current object configuration is observed again, and the active move targets are recomputed from the updated object-centric state. If the grasp verification or post-placement checking fails, the policy retries the same scheduled Hanoi move rather than advancing the symbolic move index.

Reduced real-robot evaluator. The simulator evaluator E^* returns scalar fitness and feedback metrics from simulator rollouts,

$$E^*(\mathcal{T}_{N_{\text{roll}}}(p)) = (F(p), \mathbf{M}(p)).$$

For physical deployment, a reduced evaluator E_R , distinct from the evolved E^* , computes feedback metrics only from robot execution and returns no fitness. Let $\tau_i^R(p_{\text{best}}, \kappa_i)$ denote the physical execution obtained at calibration step i with execution constants κ_i . The reduced evaluator is

$$E_R(\tau_i^R(p_{\text{best}}, \kappa_i)) = \mathbf{M}_i^R, \quad \mathbf{M}_i^R = (m_{i,1}^R, \dots, m_{i,d_R}^R).$$

The metrics in $\mathbf{M}_i^R$ are measured from robot execution and perceived object poses. They are used to calibrate the deployed controller. E_R does not introduce a new policy-search objective and is not used for selection during MEMENTO search.

LLM-assisted calibration. Calibration adjusts execution constants before the final physical evaluation. Let κ_i denote the execution constants at calibration step i , and let c_R denote the real-robot execution code. At each step, the current deployment is executed on the robot, $\mathbf{M}_i^R$ is computed by E_R , and the language model receives

$$(p_{\text{best}}, c_R, \kappa_i, \mathbf{M}_i^R).$$

The calibration proposal is

$$\kappa_{i+1} = \mathcal{L}_{\text{cal}}(p_{\text{best}}, c_R, \kappa_i, \mathbf{M}_i^R), \quad i = 0, \dots, N_{\text{cal}} - 1.$$

We use $N_{\text{cal}} = 3$ calibration steps. The final constants κ^* are selected from the tested calibration sequence and used for the final physical evaluation.

Calibrated constants. Table 4 reports the execution constants changed for physical deployment. These constants affect pick depth, approach height, container assignment tolerance, and place-hover height.

Table 4: Execution constants for physical Franka deployment. Values are in meters.

Parameter	Simulation	Physical robot
grasp_depth_offset	0.002	0.045
pick_pregrasp_clearance	0.040	0.090
peg_assign_xy	0.055	0.075
place_hover_clearance	0.110	0.100

Physical target corrections. The physical execution wrapper also applies small Cartesian corrections after the policy selects the current pick-or-place target. Pick targets are shifted by a fixed lateral centering offset. Stack-placement targets receive a fixed lateral correction when placed onto an existing stack. During the place-descend phase, the target height is lowered to seat the cube on the support surface. These corrections are part of the real execution interface and do not alter move selection, cube-to-container assignment, or move retry.

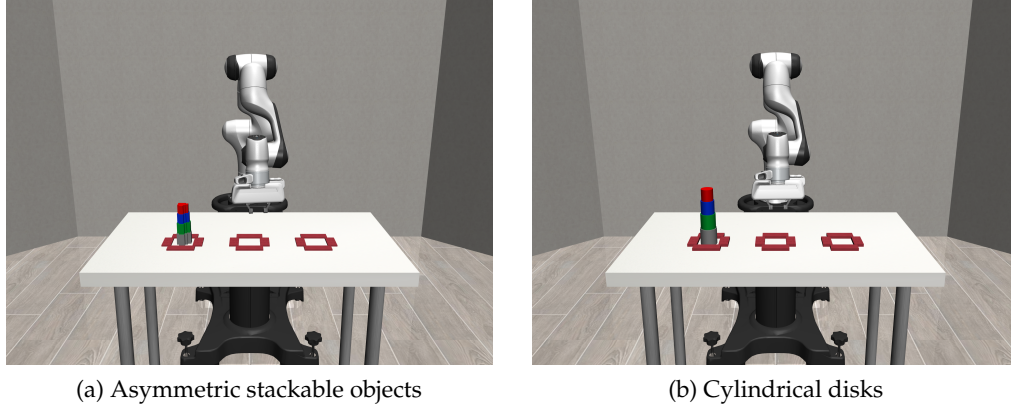


Figure 8: Held-out Robosuite Tower-of-Hanoi object variants. Both variants preserve the Hanoi task and policy observation interface used for the cubic training environment. The asymmetric-object variant changes the grasp envelope while retaining a stackable support core. The cylindrical-disk variant changes the object contact geometry by replacing cube faces with curved cylinder sides.

A.5 Held-Out Tower-of-Hanoi Object Variants

To evaluate object-geometry transfer, we use two held-out Robosuite Tower-of-Hanoi variants. Both variants preserve the Hanoi objective, object identifiers, target layout, and structured policy observation interface while changing the geometry of the manipulated objects. This tests whether the policy relies on Hanoi state relations, such as target assignment, stack order, and placement targets, rather than on the cubic object geometry used during policy search.

The asymmetric-object variant replaces each cube with a stackable central core and two protruding arms. The object remains compatible with the Hanoi stacking task, but its side surfaces and grasp points differ from those of a cube with the same scalar size. The cylindrical-disk variant replaces each cube with a cylindrical disk. This keeps flat top-bottom stacking surfaces but changes the lateral contact geometry during grasping and transport.

Table 5: Difference between the training Robosuite Tower-of-Hanoi environment and the held-out object variants.

Environment	Object geometry	Main change tested
Cube Hanoi	Symmetric cubes	Training object family
Asymmetric Hanoi	Stackable core with protruding arms	Non-cubic grasp envelope
Cylinder Hanoi	Cylindrical disks	Curved side contacts

A.6 SAC+HER Observation and Action Space

The SAC+HER baselines use the same Robosuite task and continuous low-level action space as the policy-code experiments, but their observation representation differs. The HER wrapper returns a Gymnasium dictionary observation:

$$o_t = \left\{ o_t^{\text{base}}, g_t^{\text{ach}}, g^* \right\}.$$

The base observation o_t^{base} is the flattened Robosuite object-state observation. The achieved goal g_t^{ach} is the current Cartesian position of the four cubes,

$$g_t^{\text{ach}} = [x_t^{\text{small}}, x_t^{\text{medium}}, x_t^{\text{large}}, x_t^{\text{xlarge}}] \in \mathbb{R}^{12}.$$

Table 6: Shared SAC+HER training configuration for the Robosuite Franka Tower-of-Hanoi task.

Group	Parameter	Value
Environment	Robot	Panda
Environment	Camera observations	disabled
Environment	Object observations	enabled
Environment	Control frequency	20 Hz
Environment	Episode horizon	12000 steps
Environment	Goal tolerance	0.025
SAC	Policy	MultiInputPolicy
SAC	Total timesteps	50,000,000
SAC	Learning rate	10^{-4}
SAC	Replay buffer size	2,000,000
SAC	Learning starts	25,000
SAC	Batch size	512
SAC	Discount factor γ	0.995
SAC	Target smoothing τ	0.005
SAC	Train frequency	1 step
SAC	Gradient steps	1
SAC	Entropy coefficient	auto
HER	Replay buffer	HerReplayBuffer
HER	Goal selection strategy	future
HER	Sampled goals per transition	8
Evaluation	Evaluation frequency	1,000,000 steps
Evaluation	Evaluation episodes	5

The desired goal g^* is the target tower on container C, with the extra-large cube at the bottom, followed by the large, medium, and small cubes. Quaternion goals are disabled, so both achieved and desired goals contain cube positions only.

The use of `MultiInputPolicy` follows from this dictionary observation. SAC receives `observation`, `achieved_goal`, and `desired_goal` as separate input fields. The action space is the Robosuite continuous action box exposed by the Franka environment.

A goal is successful when all four cube positions are within $\epsilon_{\text{goal}} = 0.025$ of their corresponding desired positions:

$$\text{succ}(g_t^{\text{ach}}, g^*) = \mathbb{I} \left[\max_{j \in \{1, \dots, 4\}} \|g_{t,j}^{\text{ach}} - g_j^*\|_2 < \epsilon_{\text{goal}} \right].$$

A.7 SAC+HER Training Configuration

Both SAC+HER baselines use Soft Actor-Critic with Hindsight Experience Replay. HER uses future-goal relabeling: additional goals are sampled from states achieved later in the same trajectory, and the reward is recomputed using the sparse or dense goal-conditioned reward.

Episode success, episode fitness, phase scores, milestones, and interaction-quality metrics are logged at episode termination. These metrics are used for analysis and comparison with policy-code evolution. The SAC update uses only the HER reward defined in Appendix A.8 and Appendix A.9. The SAC+HER baselines use the goal-conditioned wrapper described above; they do not use the heavy domain-randomized environment from Appendix A.3.

A.8 Sparse SAC+HER

The sparse SAC+HER baseline uses a success-only goal-conditioned reward:

$$r_{\text{sparse}}(g_t^{\text{ach}}, g^*) = \begin{cases} 0, & \text{succ}(g_t^{\text{ach}}, g^*) = 1, \\ -1, & \text{succ}(g_t^{\text{ach}}, g^*) = 0. \end{cases}$$

All unsuccessful configurations receive the same reward, independent of their distance to the target tower or their intermediate stack structure. During HER replay, relabeled transitions are scored by the same success-only criterion with respect to the sampled future goal.

A.9 Dense SAC+HER

The dense SAC+HER baseline uses the same observation representation, action space, SAC configuration, and HER relabeling method, but replaces the success-only reward with geometric shaping. Let

$$d_{t,j} = \|g_{t,j}^{\text{ach}} - g_j^*\|_2$$

be the Cartesian position error for cube j , and let

$$\bar{d}_t = \frac{1}{4} \sum_{j=1}^4 d_{t,j}$$

be the mean cube-position error. The dense reward is

$$r_{\text{dense}}(g_t^{\text{ach}}, g^*) = -\bar{d}_t + 5 \text{succ}(g_t^{\text{ach}}, g^*) - (1 - \text{legal}(g_t^{\text{ach}})).$$

The legality term checks the vertical cube ordering:

$$\text{legal}(g_t^{\text{ach}}) = \mathbb{I} \left[z_t^{\text{xlarge}} < z_t^{\text{large}} < z_t^{\text{medium}} < z_t^{\text{small}} \right].$$

The dense reward combines distance-to-target shaping, an exact success bonus, and a penalty for violating the Tower-of-Hanoi ordering constraint. During HER replay, the same dense reward is recomputed after the desired goal is replaced by the sampled future achieved goal.

A.10 Sparse–Dense Difference

The sparse and dense SAC+HER baselines differ only in the goal-conditioned reward returned by `compute_reward`. The sparse baseline uses only final goal achievement. The dense baseline adds continuous geometric progress and an ordering penalty. Both baselines use the same goal-conditioned observation representation, continuous action space, HER relabeling method, SAC architecture, training schedule, and logging metrics.

B AI2-THOR Household Environment

B.1 Task and Simulator Setup

The AI2-THOR domain is a partially observed household interaction task in kitchen scenes. The task is `apple_in_microwave_on_bread_in_fridge`. The agent must find an Apple, a Bread, a Microwave, and a Fridge; place the Apple in the Microwave, toggle the Microwave on, and place the Bread in the Fridge. Success is defined by

$$\text{apple_in_microwave} \wedge \text{microwave_on} \wedge \text{bread_in_fridge}.$$

The task is partially observed: the agent receives egocentric observations, and only currently visible objects are exposed for interaction. Object identifiers are scene-specific and must be obtained from the observation before they can be used in object actions.

B.2 AI2-THOR Code-as-Policy Interface

Code as policy evolution generates a Python class named `Policy`. The class is instantiated as `Policy(seed=0)`. Before each rollout, `reset()` clears episode-level memory. At each environment step, `compute_action(obs)` is called once and returns

$$(a_t, \iota_t),$$



Figure 9: Examples of AI2-THOR kitchen scenes used in the household interaction domain. The scenes differ in layout, object placement, visibility, and receptacle configuration.

Table 7: AI2-THOR simulator configuration.

Quantity	Value
Scenes	FloorPlan1–FloorPlan29
Image width	900
Image height	600
Visibility distance	1.5 m
Field of view	90°
Grid size	0.25 m
Maximum episode length	250 steps
Depth image	disabled
Instance segmentation	disabled

where a_t is an action name and ι_t is either an AI2-THOR object identifier or `None`. Navigation and camera actions use $\iota_t = \text{None}$. Object-interaction actions require an object identifier that is valid for the current observation.

For every object-parameterized action, the returned identifier must belong to the corresponding valid-action list:

$$\iota_t \in \text{obs}["\text{action_space}"][a_t].$$

The valid object identifiers are recomputed at every step from the simulator state, including visibility, distance, inventory state, object state, and AI2-THOR action preconditions.

B.3 Observation Space

The observation is a Python dictionary containing the current task, scene, egocentric agent state, inventory state, task-object fields, visible object summaries, task-progress predicates, valid actions, the previous action result, and step counters.

The `agent` field contains position, rotation, and camera horizon. The `inventory` field contains the held object type and held object identifier. The `targets` field reports object existence, object identifier, and visibility for Apple, Bread, Microwave, and Fridge:

```
targets = {apple_exists_in_scene, apple_object_id, apple_visible,
           bread_exists_in_scene, bread_object_id, bread_visible,
           microwave_exists_in_scene, microwave_object_id, microwave_visible,
           fridge_exists_in_scene, fridge_object_id, fridge_visible}.
```

Each entry in `visible_objects` is a summary of one currently visible object. These summaries expose the object type, exact object identifier, visibility, distance, interaction affordances, current object state, and containing receptacles.

The previous action result is part of the observation. It contains the previous action name, object identifier, success flag, normalized error category, and raw error message:

```
last_action_result = {action, object_id, success, error_category, error_message}.
```

Table 8: AI2-THOR action space exposed to generated policy code.

Policy action	Object id required	AI2-THOR action
move_ahead	no	MoveAhead
rotate_left	no	RotateLeft
rotate_right	no	RotateRight
look_up	no	LookUp
look_down	no	LookDown
open_object	yes	OpenObject
close_object	yes	CloseObject
pickup_object	yes	PickupObject
put_object	yes	PutObject
toggle_on	yes	ToggleObjectOn
toggle_off	yes	ToggleObjectOff
done	no	end episode

Table 9: Top-level AI2-THOR observation fields.

Field	Content
task	Task name
task_description	Natural-language task description
scene	Current AI2-THOR scene id
scene_is_fixed	Boolean scene flag
agent	Agent position, rotation, and camera horizon
inventory	Held object type and held object identifier
targets	Existence, identifier, and visibility fields for task objects
visible_objects	Summaries of currently visible objects
task_progress	Boolean task-progress predicates
available_actions	List of action names
action_space	Valid object identifiers for object actions
last_action_result	Previous action, success flag, and failure information
step_count	Current episode step index
max_steps	Episode step limit

The error category can indicate blocked movement, collision, a non-visible object, a non-reachable object, a failed precondition, a generic failure, or a simulator exception.

B.4 Partial Observability and Object Identifiers

The policy does not receive a complete scene graph at every step. `visible_objects` contains only objects visible from the current egocentric pose. Task-object identifiers may therefore be unavailable until the agent changes viewpoint, moves closer, or opens a receptacle. The exact `objectId` string is required for object actions, and object identifiers can differ across floor plans.

The action space is the source of validity for interactions. Even when a target object identifier is known from memory, an interaction is valid only if that identifier appears in the corresponding list in `obs["action_space"]`. This makes the policy interface state-dependent: the set of valid object actions changes with visibility, distance, object state, inventory state, and simulator preconditions.

C Additional Results

C.1 Evaluator Evolution Results

Section 3.1 defines evaluator evolution and the fixed evaluator E^* used during code-as-policy search. Here, we report the task-specific form of the selected evaluators. In both domains, E^*

Table 10: Task-progress predicates for the AI2-THOR household task.

Predicate	Meaning
apple_picked	Apple is currently held
bread_picked	Bread is currently held
microwave_open	Microwave is open
fridge_open	Fridge is open
apple_in_microwave	Apple is inside the Microwave
bread_in_fridge	Bread is inside the Fridge
microwave_on	Microwave is toggled on
success	Full task success predicate is satisfied

Table 11: Fields in each visible object summary.

Field	Content
objectType	Semantic object category
objectId	Exact identifier required for object actions
name	AI2-THOR object name when available
visible	Visibility flag
distance	Distance from the agent when available
pickupable	Whether the object can be picked up
openable	Whether the object can be opened or closed
isOpen	Current open state
receptacle	Whether the object can receive other objects
toggleable	Whether the object can be toggled
isToggled	Current toggle state
isPickedUp	Whether the object is currently picked up
parentReceptacles	Current containing receptacles, when available

returns scalar fitness for selection and feedback metrics for subsequent code-as-policy proposals:

$$E^*(\mathcal{T}_{N_{\text{roll}}}(p)) = (F(p), \mathbf{M}(p)).$$

The empirical success rate $S(p)$ is computed from the task success indicator and recorded alongside.

C.1.1 Robosuite Franka Tower-of-Hanoi

For Robosuite, the selected evaluator is defined for the four-cube Tower-of-Hanoi task with cubes

$$\mathcal{K} = \{\text{xlarge}, \text{large}, \text{medium}, \text{small}\}$$

and containers A, B, C . Let $\text{in}_j(c)$ denote that cube c lies inside the hole region of container j , and let $\text{stack}(c_1, c_2)$ denote that cube c_2 is stacked on cube c_1 with the required horizontal and vertical tolerances. The binary success predicate requires all cubes to be in the target container C , containers A and B to be empty, and the stack on C to satisfy the legal Hanoi order:

$$\begin{aligned} \text{success}_e = & \left(\bigwedge_{c \in \mathcal{K}} \text{in}_C(c) \right) \wedge \left(\bigwedge_{c \in \mathcal{K}} \neg \text{in}_A(c) \wedge \neg \text{in}_B(c) \right) \\ & \wedge \text{stack}(\text{xlarge}, \text{large}) \wedge \text{stack}(\text{large}, \text{medium}) \wedge \text{stack}(\text{medium}, \text{small}). \end{aligned}$$

The task success indicator is

$$S_e = \mathbb{I}\{\text{success}_e\},$$

computed from the environment success predicate. The evaluator also returns a dense episode fitness $f_e \in [0, 1]$. The policy-level fitness is the mean episode fitness over the rollout budget:

$$F(p) = \frac{1}{N_{\text{roll}}} \sum_{e=1}^{N_{\text{roll}}} f_e(p).$$

At each control step t , the evaluator computes phase scores for approach, grasp/lift, transport, placement, and final arrangement. The approach score is the maximum proximity score between the end effector and any cube:

$$A_t = \max_{c \in \mathcal{K}} \exp\left(-\frac{\|x_{\text{eef},t} - x_{c,t}\|}{0.12}\right).$$

The grasp/lift score is the maximum normalized lift height:

$$G_t = \max_{c \in \mathcal{K}} \text{clip}_{[0,1]} \left(\frac{z_{c,t} - (z_{\text{table}} + s_c)}{0.08} \right),$$

where s_c is the cube half-edge length. The transport score rewards lifting a cube while moving it toward the target container:

$$T_t = \max_{c \in \mathcal{K}} \ell_{c,t} \exp\left(-\frac{d_C(c,t)}{0.18}\right),$$

where $\ell_{c,t}$ is the normalized lift score of cube c , and $d_C(c,t)$ is the planar error between cube c and the target hole center in container C .

Placement is measured using continuous hole scores. Let $h_j(c,t) \in [0, 1]$ be the score that cube c lies inside the hole of container j . The target-fill and source-emptiness terms are

$$C_{\text{fill},t} = \frac{1}{4} \sum_{c \in \mathcal{K}} h_C(c,t),$$

$$E_{AB,t} = \frac{1}{2} \left(1 - \frac{1}{4} \sum_{c \in \mathcal{K}} h_A(c,t) + 1 - \frac{1}{4} \sum_{c \in \mathcal{K}} h_B(c,t) \right).$$

The placement score is

$$L_t = \text{clip}_{[0,1]} (0.75 C_{\text{fill},t} + 0.25 E_{AB,t}).$$

The arrangement score evaluates the target stack on container C :

$$R_t = \text{clip}_{[0,1]} \left(0.50 \bar{g}_t + \frac{1}{6} \sigma_{\text{xlarge,large},t} + \frac{1}{6} \sigma_{\text{large,medium},t} + \frac{1}{6} \sigma_{\text{medium,small},t} \right),$$

where $\bar{g}_t$ is the mean goal-pose score over the four cubes and $\sigma_{i,j,t}$ is the continuous score that cube j is stacked on cube i .

The per-step composite score is

$$\tilde{P}_t = 0.14 A_t + 0.18 G_t + 0.20 T_t + 0.20 L_t + 0.28 R_t.$$

Illegal Hanoi placements are penalized by the maximum continuous score for a larger cube being placed on a smaller cube:

$$P_t = \text{clip}_{[0,1]} \left(0.95 \tilde{P}_t (1 - 0.15 U_t) + 0.05 Q_{\text{int},t} \right),$$

where $U_t \in [0, 1]$ is the illegal-stack score and $Q_{\text{int},t}$ measures interaction smoothness from end-effector and cube speeds.

For an episode ending at step T_e , the evaluator records the time-average composite score, the best composite score, and the final arrangement score:

$$P_{\text{auc}} = \frac{1}{T_e} \sum_{t=1}^{T_e} P_t, \quad P_{\text{best}} = \max_{1 \leq t \leq T_e} P_t, \quad R_{\text{final}} = R_{T_e}.$$

The episode fitness is

$$f_e = \text{clip}_{[0,1]} (0.45 P_{\text{auc}} + 0.45 P_{\text{best}} + 0.10 R_{\text{final}}).$$

If the Tower-of-Hanoi success predicate is satisfied, f_e is set to 1.0.

The feedback metrics returned by the evaluator include maximum and final phase scores for approach, grasp/lift, transport, placement, and arrangement; milestone times for first reach, first lift, first cube in C , first two cubes in C , and all cubes in C ; minimum end-effector-cube distances; maximum lift heights; cube transport speeds; hole scores for containers A , B , and C ; target-fill and source-emptiness scores; goal-pose scores; legal stack-pair scores; drop count; and maximum illegal-stack score. These feedback metrics localize failure to reaching, lifting, transport, placement, final stack arrangement, illegal Hanoi placement, or unstable interaction, and are included with scalar fitness when generating subsequent code-as-policy proposals.

C.1.2 AI2-THOR Household Interaction

For AI2-THOR, the selected evaluator is defined for the task `apple.in.microwave.on.bread.in.fridge`. The binary success predicate is the same as in Appendix B.1:

$$\text{success}_e = \text{apple.in.microwave} \wedge \text{microwave.on} \wedge \text{bread.in.fridge}.$$

The task success indicator is

$$S_e = \mathbb{I}\{\text{success}_e\},$$

computed from the environment success predicate. The evaluator returns a dense episode fitness $f_e \in [0, 1]$. The policy-level fitness is the mean episode fitness over the rollout budget:

$$F(p) = \frac{1}{N_{\text{roll}}} \sum_{e=1}^{N_{\text{roll}}} f_e(p).$$

At each environment step t , the evaluator computes a phase-progress score P_t from the current task predicates and accumulated visibility history:

$$\begin{aligned} P_t = & 0.06 p_{\text{apple.seen},t} + 0.06 p_{\text{bread.seen},t} + 0.08 p_{\text{apple.picked},t} + 0.08 p_{\text{bread.picked},t} \\ & + 0.10 p_{\text{microwave.seen},t} + 0.10 p_{\text{fridge.seen},t} + 0.08 p_{\text{microwave.open},t} + 0.08 p_{\text{fridge.open},t} \\ & + 0.16 p_{\text{apple.in.microwave},t} + 0.16 p_{\text{bread.in.fridge},t} + 0.04 p_{\text{microwave.on},t}. \end{aligned}$$

Visibility terms are fractional and depend on whether the object has been observed and, when visible, its distance to the agent. State-change terms such as pickup, receptacle opening, object insertion, and toggling are binary once the corresponding predicate is reached.

For an episode ending at step T_e , the evaluator records

$$P_{\text{best}} = \max_{0 \leq t \leq T_e} P_t, \quad P_{\text{final}} = P_{T_e}.$$

The progress term used in the episode fitness is

$$P = 0.35 P_{\text{best}} + 0.65 P_{\text{final}}.$$

Thus, P_{best} gives credit for the best intermediate task state reached during the rollout, while P_{final} measures the task progress that remains at termination.

The final episode fitness is

$$f_e = \text{clip}_{[0,1]}(0.74 P + 0.14 Q_{\text{int}} + 0.06 Q_{\text{vis}} + 0.06 Q_{\text{eff}} - 0.07 C_{\text{blocked}} - 0.03 C_{\text{carry}}).$$

Here Q_{int} measures valid and task-relevant object interactions, Q_{vis} measures visibility of the Apple, Bread, Microwave, and Fridge, and Q_{eff} rewards earlier progress within the episode horizon. The penalty C_{blocked} increases with failed movement and long intervals without progress. The penalty C_{carry} increases when the policy holds the Apple or Bread for many steps without placing it in the required receptacle.

The evaluator also enforces lower bounds on the milestones of f_e . Any successful Apple or Bread pickup provides at least 0.06 fitness. Observing the Microwave or Fridge gives at least 0.10. Opening the Microwave or Fridge gives at least 0.18. Placing either the Apple in the Microwave or the Bread in the Fridge gives at least 0.42. Completing both placement predicates gives at least 0.72. Completing both placement predicates and turning on the Microwave gives at least 0.90. Full task success gives at least 0.98.

The feedback metrics returned by the evaluator include terminal task predicates, best- and final-phase scores, first-hit times for task milestones, visibility and proximity statistics for all target objects, action success rates, task-relevant interaction success rates, failed movement rate, and carrying-stall measures. The milestone times record when the policy first picks up the Apple or Bread, first observes the Microwave or Fridge, opens each receptacle, places each object in its target receptacle, and toggles the Microwave on. These feedback metrics identify the unsatisfied AI2-THOR precondition at the level used by the code-as-policy interface: object discovery, valid pickup, receptacle localization, receptacle opening, object placement, Microwave toggling, or premature termination.

C.2 Evolved Code-as-Policy Programs

We report one successful evolved code-as-policy program from each domain to illustrate the executable strategy produced by policy evolution. Since multiple final policies solve the task, the discussion focuses on the policy structure: task-state representation, control-flow logic, closed-loop execution, and recovery mechanisms.

C.2.1 Robosuite Franka Tower-of-Hanoi

The Franka code-as-policy program combines a fixed symbolic Hanoi schedule with guarded closed-loop execution. The symbolic component generates the recursive 15-move solution for four cubes and three containers. The policy state is

$$(i, z, r),$$

where i is the current Hanoi move index, z is the manipulation state, and r is the retry index. The move index is advanced only after the corresponding physical placement passes a post-placement check.

At each control step, the policy filters the observed end-effector pose, cube poses, and container poses using separate exponential moving averages. It then computes the three container hole centers from the current container poses. For the active Hanoi move, the pick target is the observed pose of the selected cube plus a retry-dependent planar offset. The target place is the observed destination hole center with a height determined from the current top surface of the destination stack, excluding the cube being moved.

The manipulation controller is implemented as a finite-state machine with pick, grasp-verification, placement, centering, release, and retreat states. In the pick stage, the policy moves to a travel height above the selected cube, descends to a pre-grasp pose, and closes the gripper. In the grasp-verification stage, the policy lifts the cube and checks whether it has moved upward from its grasp-time pose while remaining close to the end effector. In the placement stage, the policy moves above the destination container and descends only once it is horizontally aligned. In the centering and release states, the policy centers on the target hole at contact height, then opens the gripper. After release, the policy checks whether the cube is near the destination hole before advancing the move index.

Recovery is local to the active Hanoi move. If grasp verification fails, the gripper is opened, the retry index is incremented, and the same Hanoi move is attempted again with a different

planar pick offset. If the post-placement check fails, the move index is not advanced, so the same Hanoi move is retried from the updated observation. Thus, the symbolic Hanoi ordering remains fixed, but the execution of each move is adapted through retry-conditioned targets.

The controller also uses contact-aware motion constraints. Downward motion is suppressed until the horizontal error is below the corresponding pick or place gate. Cartesian commands are slew-limited, and lateral motion is reduced near the stack to avoid disturbing previously placed cubes. Before release, a centering state applies small lateral corrections until the residual planar error and commanded lateral motion are both small. The largest cube uses a tighter placement gate and a longer centering window, since its placement forms the base of the final tower.

C.2.2 AI2-THOR Household Interaction

The selected AI2-THOR code-as-policy program solves the task by using observation-conditioned object interactions rather than a fixed action sequence. The policy maintains memory of object identifiers and parsed object positions for the Apple, Bread, Microwave, and Fridge. At each step, it selects object identifiers only from the current `obs["action_space"]`, so pickup, open, put, and toggle actions are issued only when the simulator exposes them as executable.

Navigation uses the agent grid pose and remembered object positions. The policy records failed `move_ahead` attempts as blocked grid edges and avoids them in subsequent navigation. When direct movement toward a target object is obstructed, it switches to an axis-aligned greedy choice over neighboring grid cells, using blocked-edge memory and container-specific bad-placement cells to avoid repeating failed poses.

The main domain-specific recovery mechanism is for receptacle placement. If `put_object` fails at the Microwave or Fridge, the policy marks the current grid cell as a bad placement pose for that receptacle and executes a short pose-adjustment sequence before retrying the placement from a different viewpoint. Separate recovery queues are used for Microwave and Fridge placement failures, reflecting the different geometry and visibility constraints of the two receptacles.

The policy also handles the hidden-Apple case. If the Apple is not found during a normal search, the policy probes the Fridge, opens it when a valid Fridge identifier is available, scans the open receptacle with camera actions, and attempts to pick up the Apple from inside. If the Apple is picked from an open Fridge, the policy closes the Fridge before navigating to the Microwave, avoiding collisions and failed navigation caused by the open door.